



APRENDIZADO PROFUNDO

deep learning

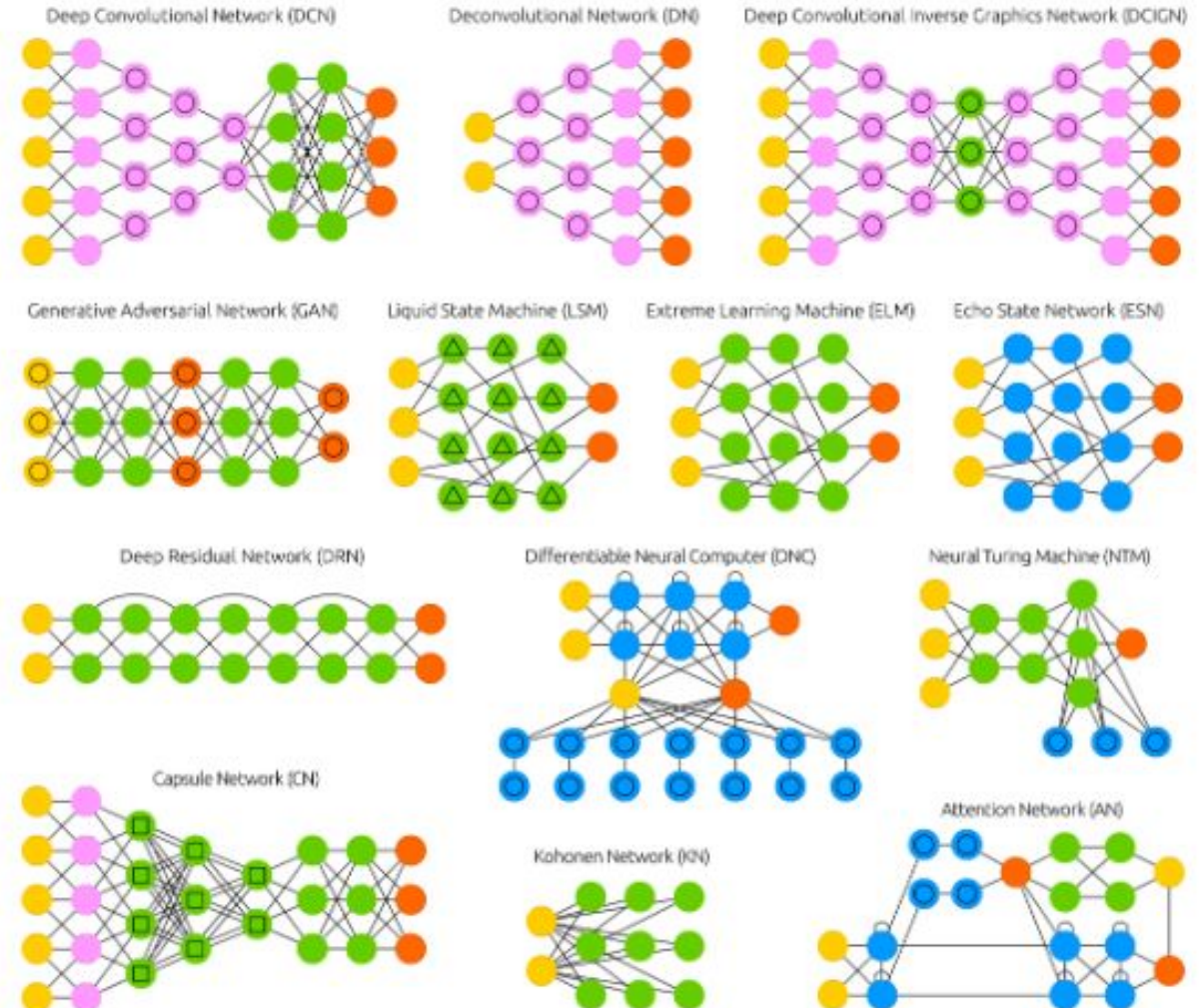
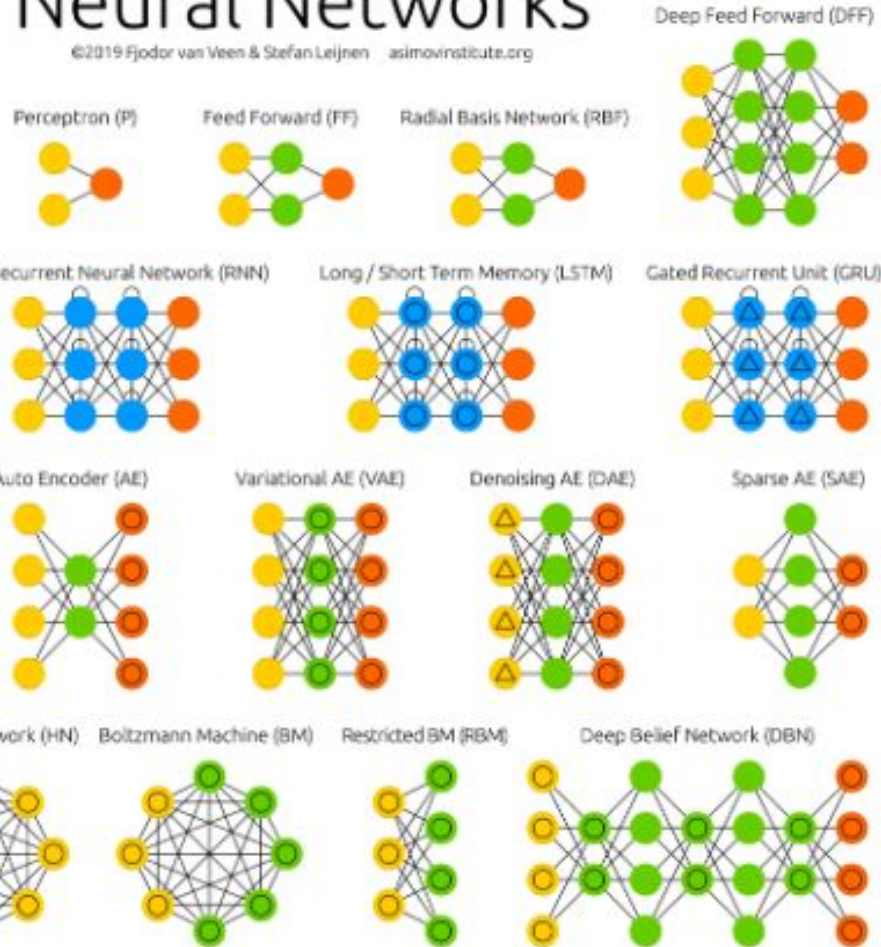
PROF. JOSENALDE OLIVEIRA

PROGRAMA DE PÓS-GRADUAÇÃO EM TI (PPgTI) - UFRN

A mostly complete chart of Neural Networks

©2019 Fjodor van Veen & Stefan Leijnen asimovinstitute.org

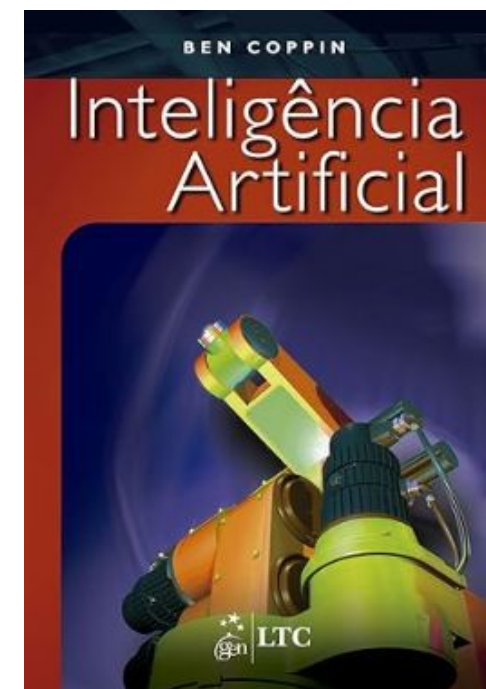
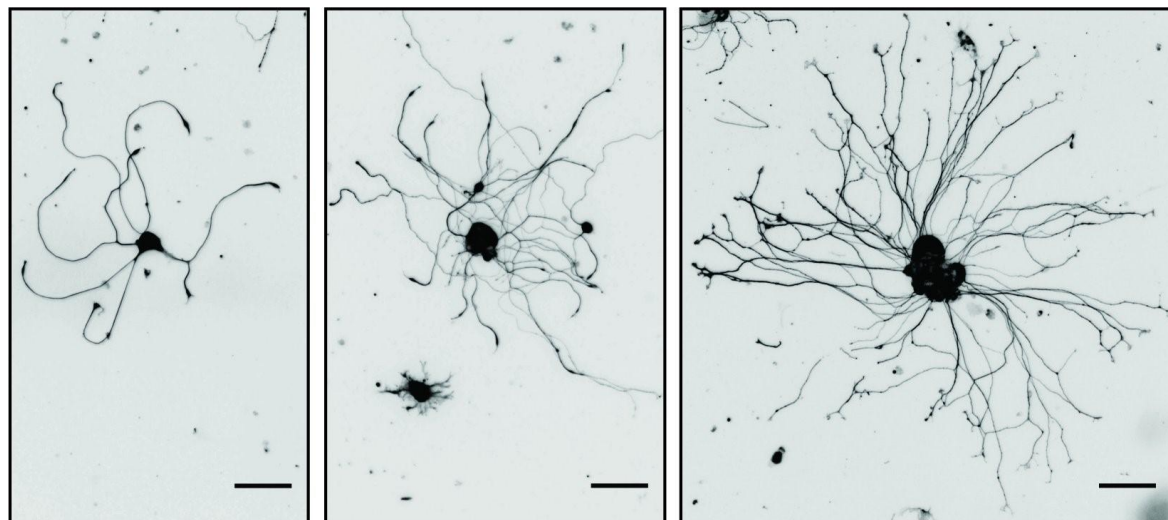
-  Input Cell
-  Backfed Input Cell
-  Noisy Input Cell
-  Hidden Cell
-  Probabilistic Hidden Cell
-  Spiking Hidden Cell
-  Capsule Cell
-  Output Cell
-  Match Input Output Cell
-  Recurrent Cell
-  Memory Cell
-  Gated Memory Cell
-  Kernel
-  Convolution or Pool



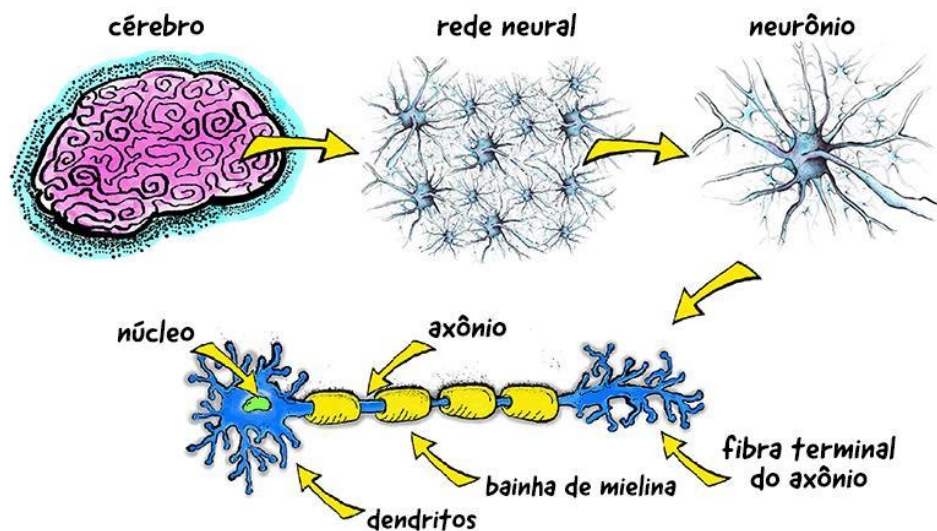
TUDO ENTÃO PASSAR POR ENTENDER REDES NEURAIS ARTIFICIAIS

Objetivo: emular componentes, estrutura, processamento neuronal de informações com base nas entradas sensoriais, produzindo saídas/estímulos e aprendizado. Dado o elemento base ser o neurônio, é necessário um **neurônio artificial**

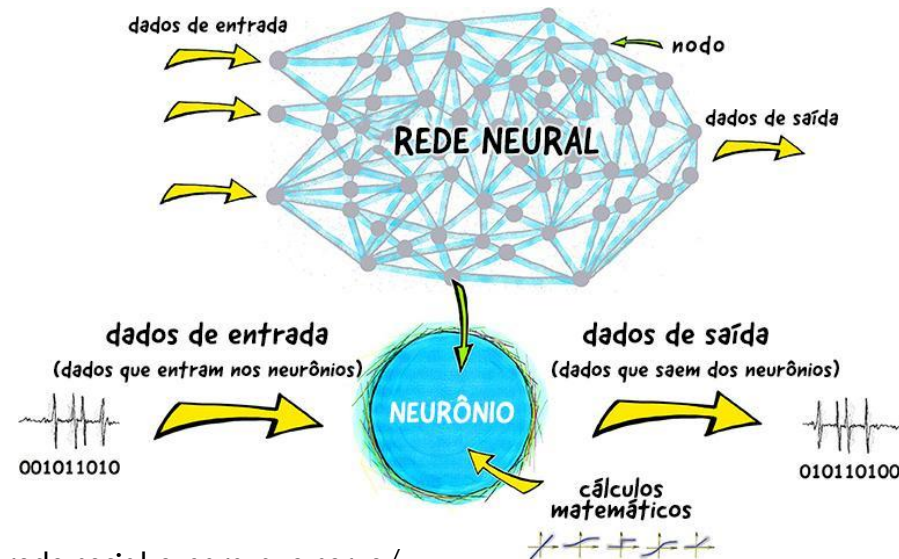
*“O cérebro humano possui bilhões de neurônios, cada qual combinado, em média, a milhares de outros neurônios. Estas conexões são conhecidas por **sinapses** e o cérebro humano possui cerca de 60[+] trilhões destas conexões” (Cobbin, 2010, Cap.11)*



REDES NEURAIS ARTIFICIAIS



<https://parajovens.unesp.br/o-que-e-uma-rede-social-e-para-que-serve/>



Funcionamento do neurônio:

- Cada neurônio é composto por um **corpo**, também chamado de soma, um **axônio** e vários **dendritos**
- Os **dendritos** tem por função receber impulsos nervosos dos outros neurônios e conduzi-los ao corpo do neurônio (ENTRADA)
- O **corpo** do neurônio processa os impulsos recebidos e gera um novo impulso nervoso (PROCESSADOR)
- Se esse novo impulso gerado for suficientemente alto, ele será repassado para outros neurônios através do **axônio**, **em cuja terminação estão as sinapses** (SAÍDA)

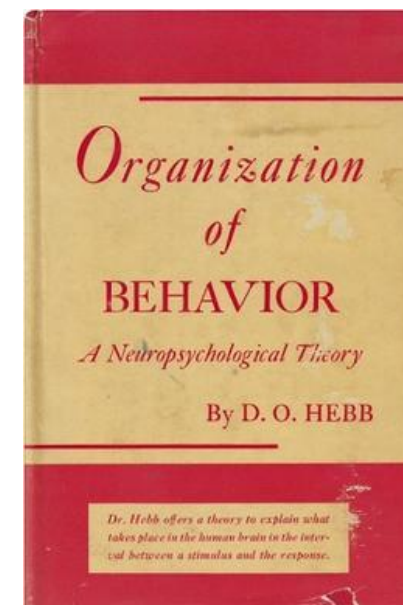
REDES NEURAIS ARTIFICIAIS

Plasticidade: os neurônios podem modificar a natureza de suas conexões em resposta a eventos novos, ou seja, REFORÇAR conexões que levem a soluções corretas do problema e ENFRAQUECER conexões que levem a soluções incorretas

A APRENDIZAGEM é o resultado destas conexões adaptáveis. As conexões ESTÁVEIS bases de memórias duradouras não são aparentemente afetadas pelo aprendizado

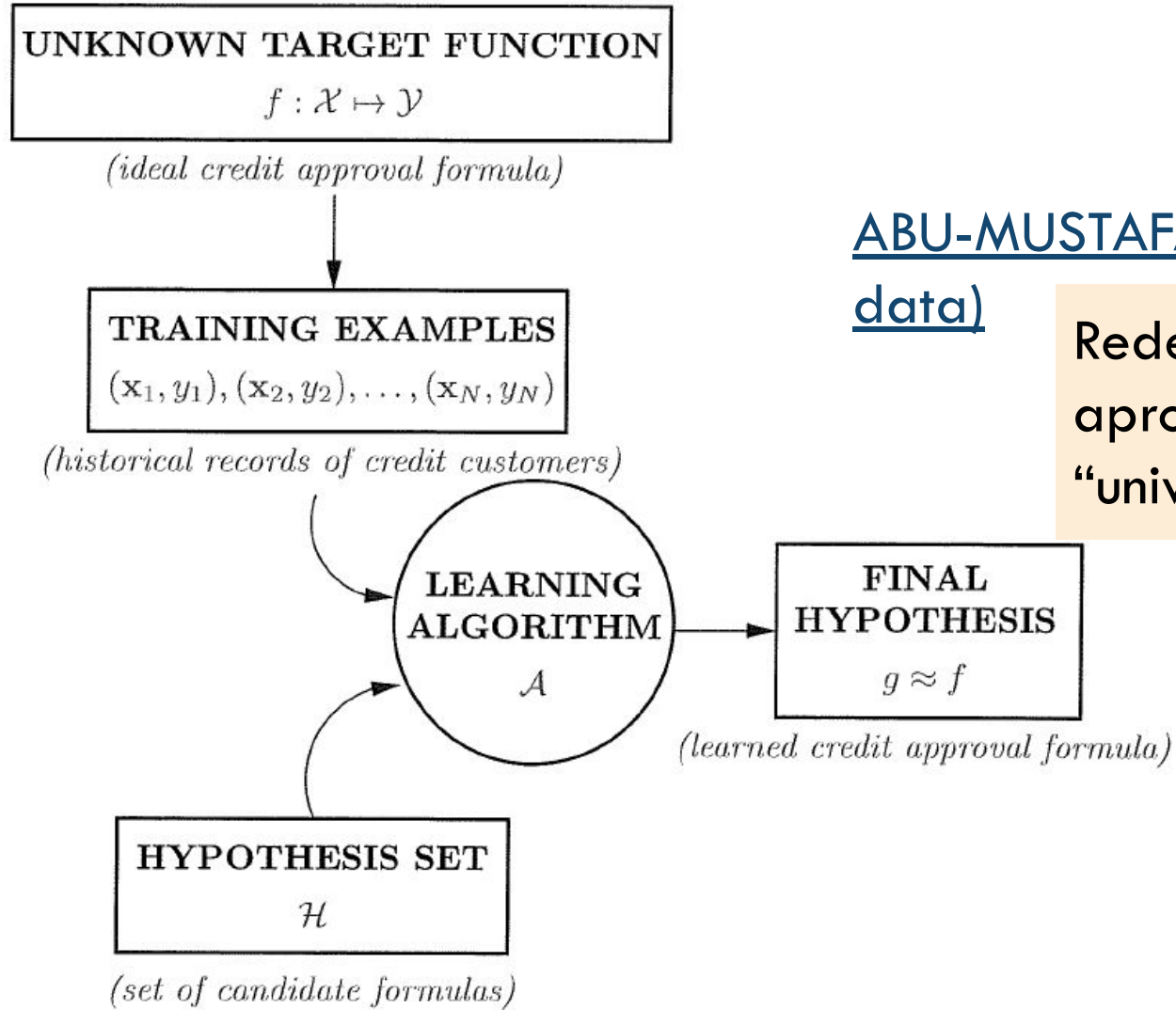
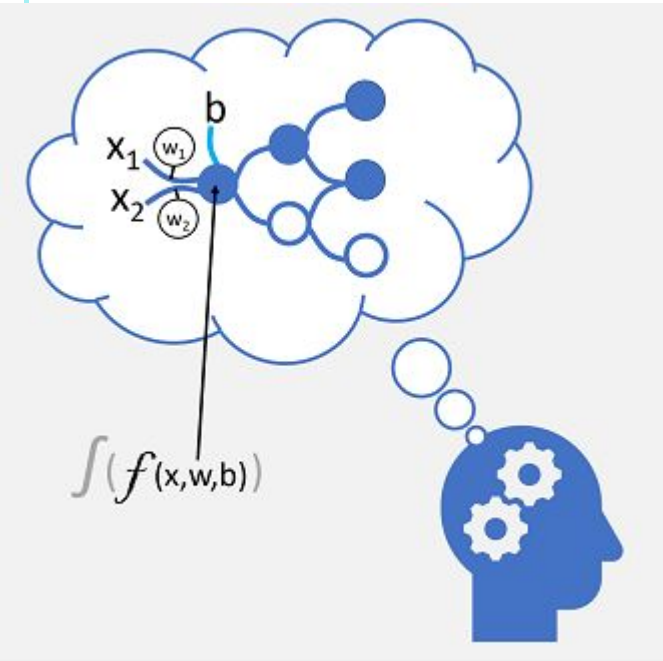


A aprendizagem hebbiana é uma teoria na neurociência que explica como as conexões neurais (sinapses) se fortalecem. É frequentemente resumida pela frase: "Células que disparam juntas, conectam-se juntas"



The Organization of Behavior (1949)

PRINCÍPIOS DE COMO SE APRENDE A PARTIR DE DADOS

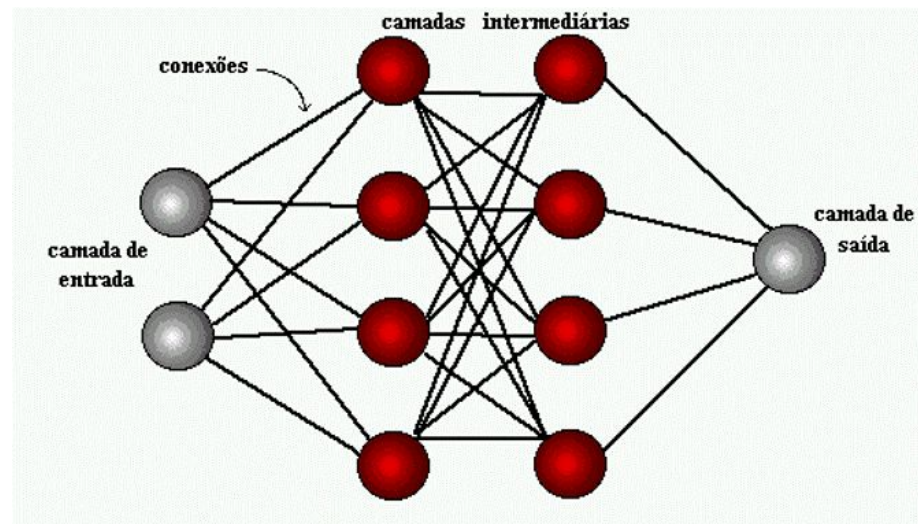


ABU-MUSTAFA et al. (learning from data)

Redes neurais como aproximadores “universais” de funções

Figure 1.2: Basic setup of the learning problem

NEURÔNIO ARTIFICIAL, CAMADAS, REDE



Cada neurônio artificial calcula uma função matemática, como um nó de processamento em um grafo computacional

Estão organizados em camadas (*layers*), com os estímulos percorrendo no sentido *forward* (*input-hidden-output*) – *feedforward* (redes recorrentes *RNN* se comportam de maneira diferente)

Estas conexões - **axônio-sinapse-dendritos no biológico e arestas do grafo computacional** – possuem PESOS (*weights*) associados, ponderando a entrada recebida do neurônio (11 neurônios, 28 conexões no exemplo acima) $= (2 \times 4) + (4 \times 4) + (4 \times 1)$
Quando cada neurônio da camada K está ligado a todos os neurônios da camada $K+1$, dizemos que a camada K é **DENSA**.

A configuração de pesos em determinado momento T (estado) define o aprendizado da rede no instante T

NEURÔNIO ARTIFICIAL, CAMADAS, REDE

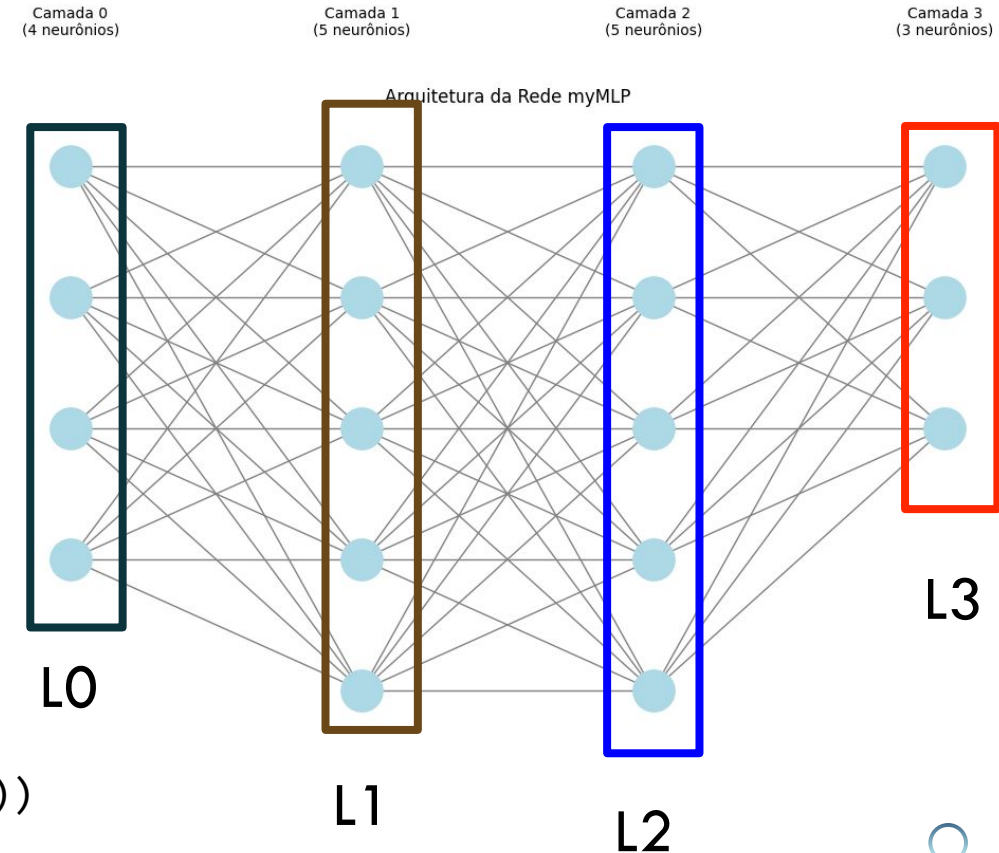
#Exemplo 1

```
import torch
import torch.nn as nn

class myMLP(nn.Module):
    def __init__(self):
        super(myMLP, self).__init__()
        self.layer1 = nn.Linear(4, 5)
        self.layer2 = nn.Linear(5, 5)
        self.layer3 = nn.Linear(5, 3)

    def forward(self, x):
        layer1_output = torch.relu(self.layer1(x))
        layer2_output = torch.relu(self.layer2(layer1_output))
        y = self.layer3(layer2_output)
        return y

# Move the model to the GPU
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
net = myMLP().to(device)
```



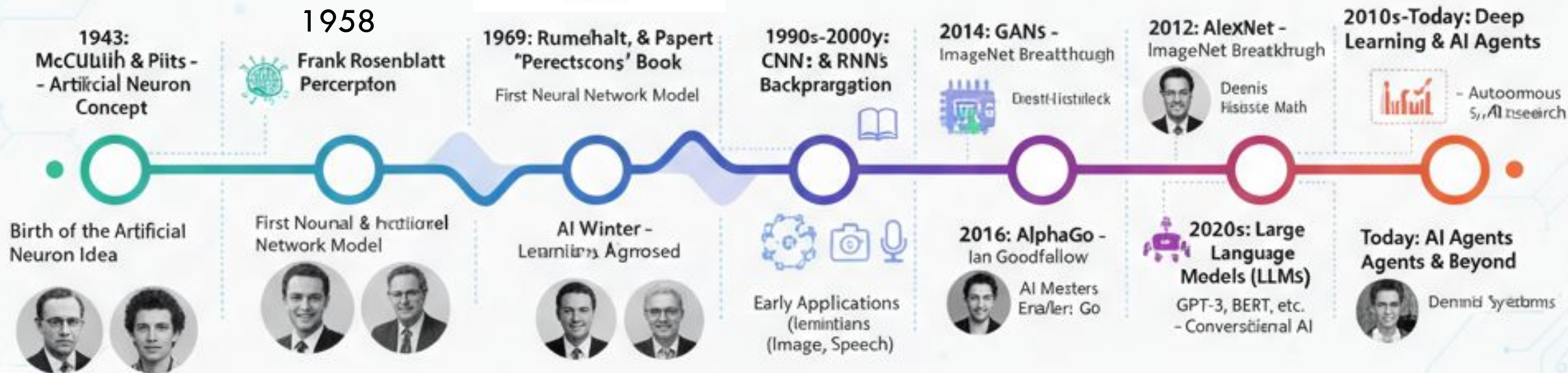
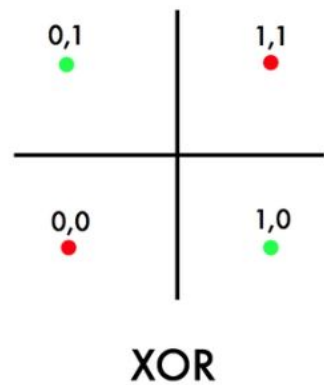
TRANSFER LEARNING - TRANSFERINDO PESOS

Como Funciona a Transferência de Pesos

O processo geralmente envolve os seguintes passos:

- **Pré-treino:** Uma rede neural (frequentemente uma Rede Neural Convolutacional para tarefas de visão computacional) é treinada numa tarefa genérica e vasta, como a classificação de milhares de categorias de imagens no conjunto de dados [ImageNet](#). Durante este treino, a rede aprende a extrair características genéricas úteis (como bordas, texturas e formas).
- **Transferência:** Os pesos aprendidos nesta fase são copiados e usados como ponto de partida (inicialização) para uma nova rede que será treinada numa tarefa específica diferente, como a classificação de raças de cães.
- **Ajuste Fino (*Fine-Tuning*):** Os pesos transferidos são então ajustados (treinados) usando os novos dados específicos da tarefa para otimizar o desempenho no novo problema [2]. ([PEFT](#), [LoRA](#), [qLoRA](#))

TIMELINE



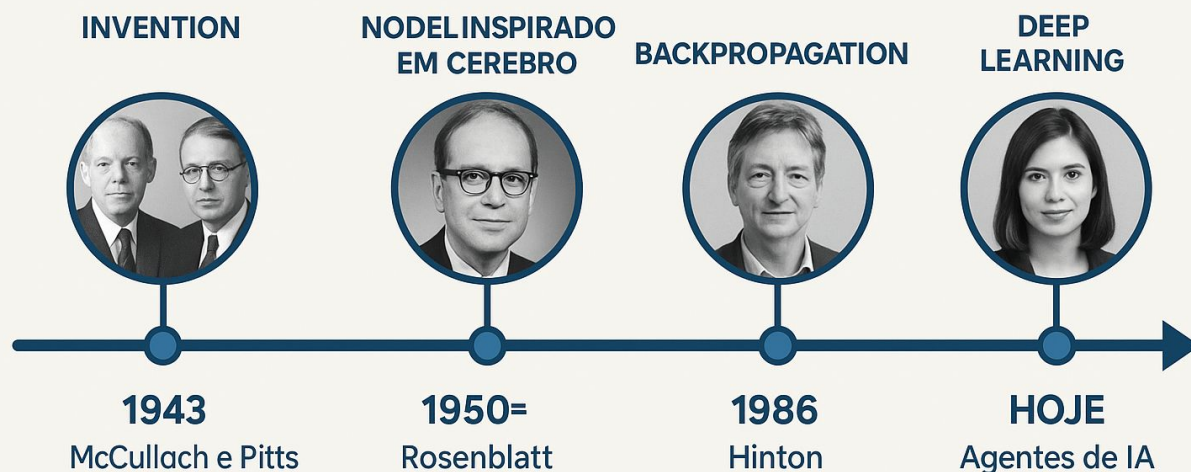
Fonte: gerada por IA* a partir de prompt do autor

*gemini flash 2.5

TIMELINE

<https://hal.science/hal-04206682/file/Lecun2015.pdf>

HISTÓRIA DAS REDES NEURAIS



Fonte: gerada por IA* a partir de prompt do autor

*dall-e (openai)

Deep Learning (2015)

Transformer (2017)

CapsNets (2017-Hinton)

Vision Transformer (2020)

Nested Learning (2025)

Kolmogorov-Arnold Network
(KAN) 2025

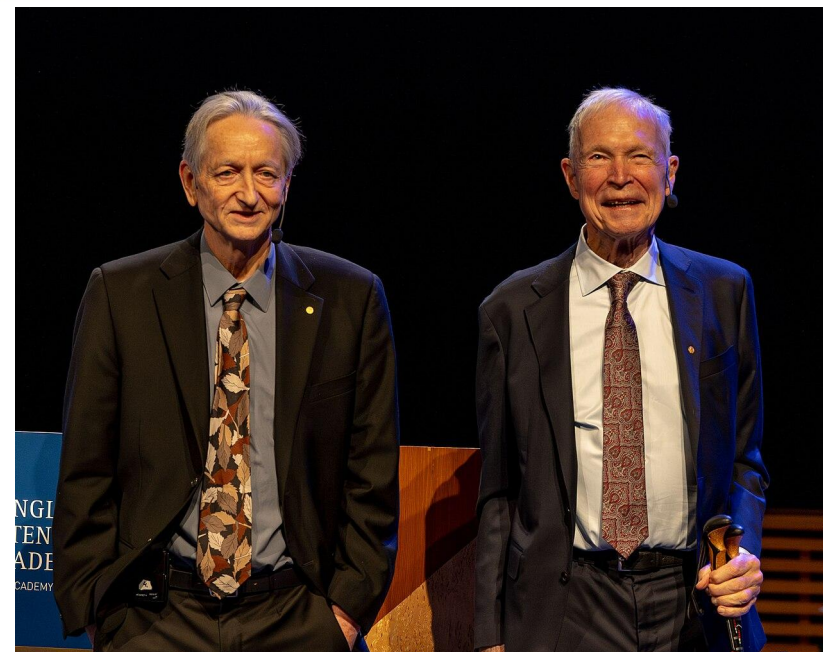
TIMELINE - NOBEL FÍSICA 2024

- **John Hopfield:** Desenvolveu as redes de Hopfield em 1982, que demonstraram como as redes neurais poderiam aprender e reter informações, utilizando conceitos da física, como a energia de sistemas de spin.
- **Geoffrey Hinton:** Expandiu esses conceitos ao desenvolver a máquina de Boltzmann, inspirada na física estatística, que ajudou a criar algoritmos para treinar redes neurais. Ele também é conhecido por seu trabalho no algoritmo de "backpropagation" (retropropagação).

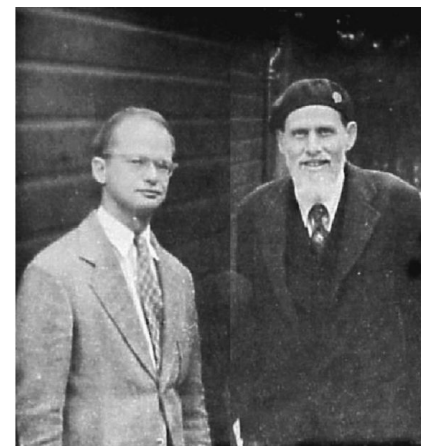
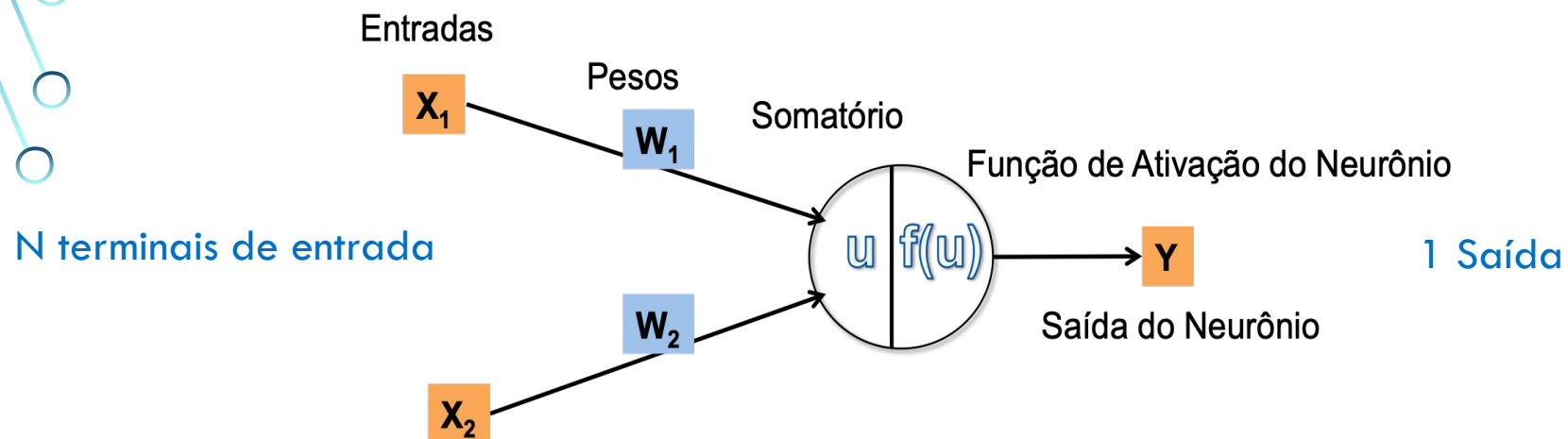
Redes neurais
associativas



*foundational discoveries and inventions that
enable machine learning with artificial neural
networks*



NEURÔNIO MCP – O INÍCIO...



1943: Warren McCulloch (neurofisiologista, Universidade Illinois, Chicago, 1898-1969) e Walter Pitts (psicólogo cognitivo, lógica, auto didata, 1923-1969)

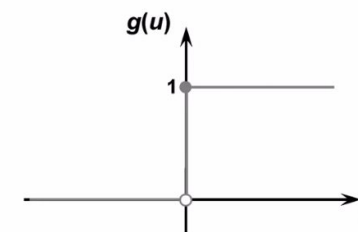
$$u = \sum_{i=1}^N x_i w_i$$

f é a função de ativação do neurônio (se permite ou não o sinal passar). É necessário que o sinal u ultrapasse um limiar

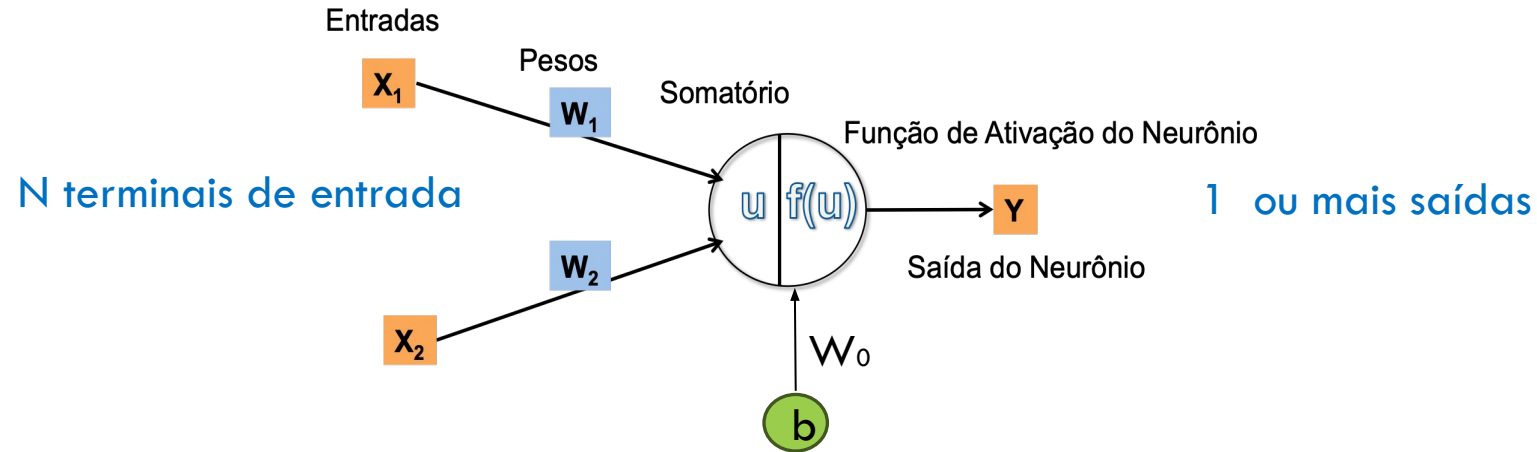
Neste caso as entradas são binárias, bem como saída, podendo ser combinados neurônios para executar qualquer lógica proposicional. Nas primeiras ideias, um neurônio é ATIVO se ao menos duas de suas entradas forem ATIVAS.

heaviside

Função degrau $\rightarrow g(u) = \begin{cases} 1 & \text{se } (u \geq 0) \\ 0 & \text{se } (u < 0) \end{cases}$



EVOLUINDO PARA PERCEPTRON...



1957: Frank Rosenblatt (neurônio chamado Unidade de Limiar Linear (LTU) ou simplesmente Perceptron)

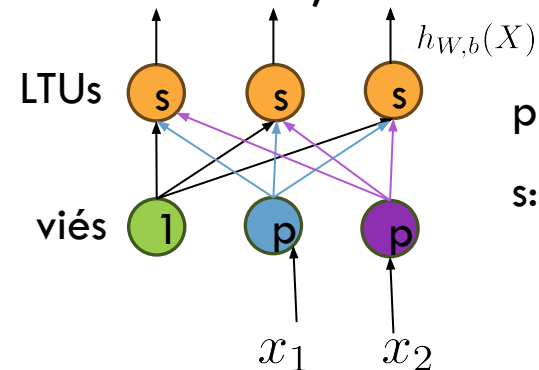
As entradas e saídas são números quaisquer, com pesos quaisquer

$$u = \sum_{i=1}^N x_i w_i, \quad f(u) = \text{heaviside}(u) \text{ ou } f(u) = \text{sgn}(u)$$

Esta arquitetura é a mais simples possível, possuindo uma única camada de LTUs, e cada LTU está conectada a todas as entradas. Pode ser um classificador multiclasse

$$h_{W,b}(X) = \phi(XW + b)$$

Transformação Afim



p: neurônio de entrada (passagem)
s: somador + função ativação (degrau)

REGRA DE APRENDIZADO NA PERCEPTRON

Rosenblatt propõe algoritmo de treinamento inspirado na regra de Hebb (1949): reforça as conexões (aumenta peso) de conexões que ajudam a reduzir o erro de predição

Injeta-se na rede uma instância de treinamento $\mathbf{x}$ por vez, e para cada instância, faz predições.

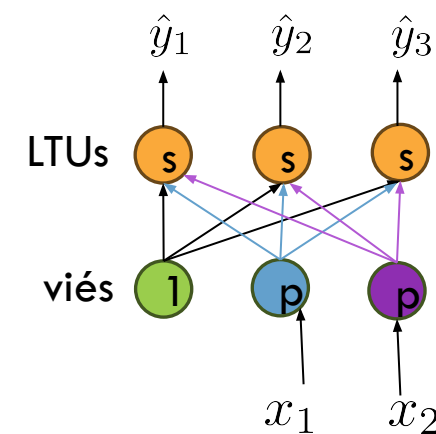
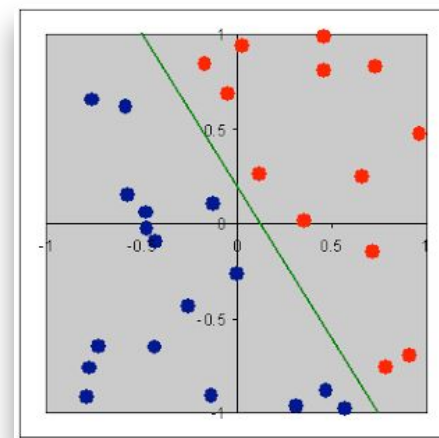
Para cada neurônio de saída que gerou predição errada, reforça os pesos de conexão das entradas que teriam contribuído para a predição correta. Na equação abaixo, i (entrada), j (saída)

$$w_{i,j}(k+1) = w_{i,j}(k) + \eta(y_j - \hat{y}_j)x_i \quad \text{Parece um SGD}$$

Fronteira de decisão de cada saída é LINEAR

Logo não aprende padrões complexos (não lineares)

Instâncias de treinamentos precisam ser linearmente separáveis



REGRA DE APRENDIZADO NA PERCEPTRON

```
import numpy as np
from sklearn.datasets import load_iris

df = load_iris()
X = df.data[:, (2,3)] #petal length, petal width (x1, x2)
y = (df.target == 0).astype(np.int32) #setosa ?
from sklearn.linear_model import Perceptron
model = Perceptron() #equivale a SGDClassifier(loss=perceptron,
learning_rate=constant, eta0=1, penalty=None)
model.fit(X,y)

y_hat = model.predict([[2,0.5]])
print(y_hat) #0 (não gera probabilidades, como a regressão logística!)
```

Transforma em
problema binário

EXEMPLO SIMPLES: RNA RECONHECENDO PORTA AND

$Y = 1$ (se X maior ou igual a 1)

$Y = 0$ (se X menor que 1)

A	B	W1	W2	Porta E	Y (rede)
0	0			0	
0	1			0	
1	0			0	
1	1			1	

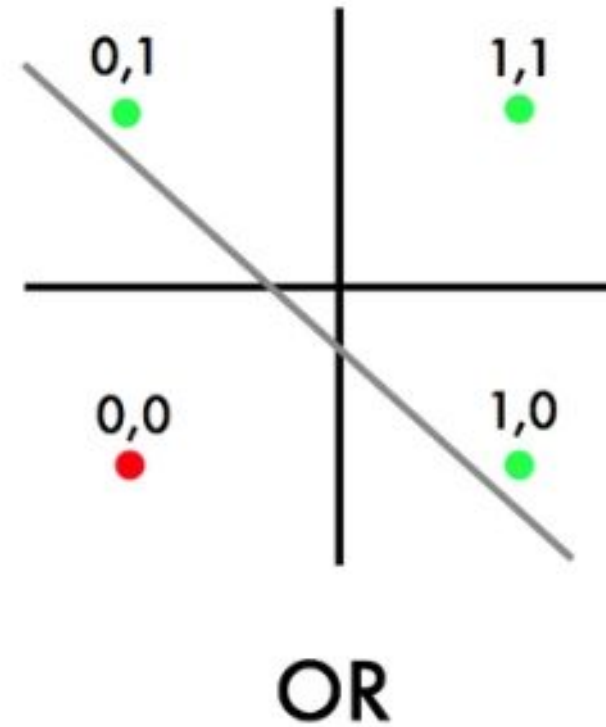
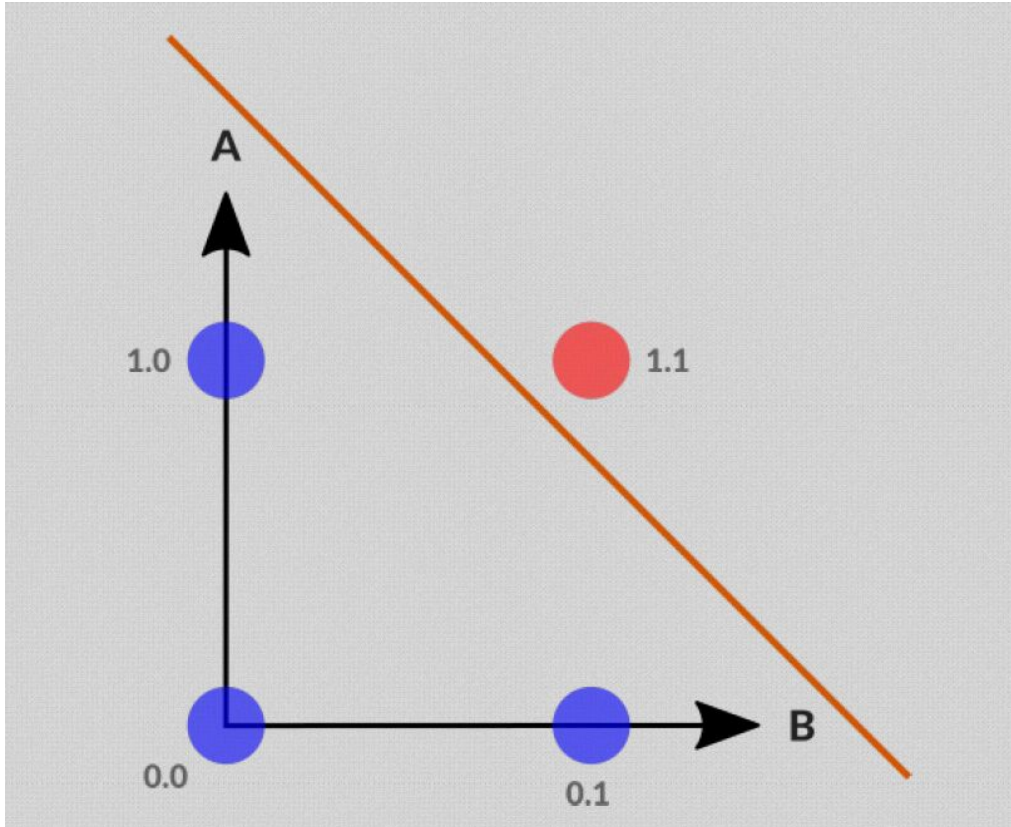
EXEMPLO SIMPLES: RNA RECONHECENDO PORTA AND

A	B	W1	W2	Porta E	Y (rede)
0	0	0.1	0.3	0	0; Y=0
0	1	0.1	0.3	0	0.3; Y=0
1	0	0.1	0.3	0	0.1; Y=0
1	1	0.1	0.3	1	0.4; Y=0

EXEMPLO SIMPLES: RNA RECONHECENDO PORTA AND

A	B	W1	W2	Porta E	Y (rede)
0	0	0.8	0.3	0	0; Y=0
0	1	0.8	0.3	0	0.3; Y=0
1	0	0.8	0.3	0	0.8; Y=0
1	1	0.8	0.3	1	1.1; Y=1

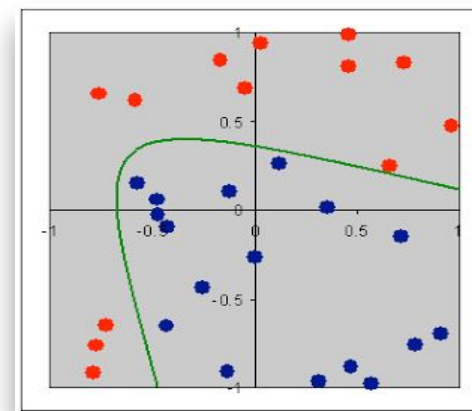
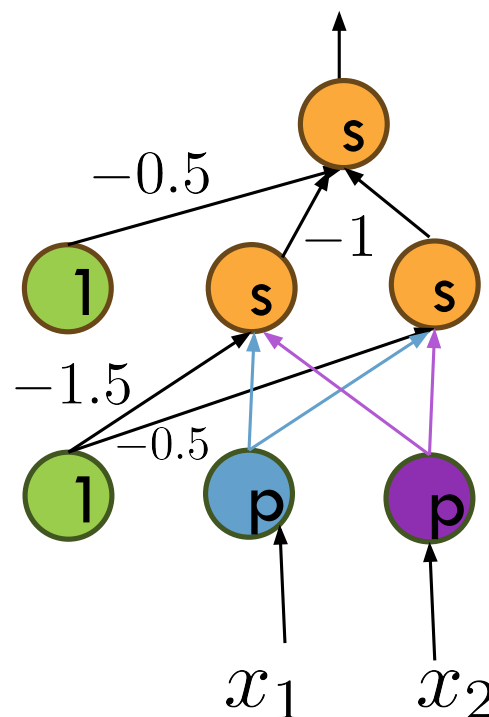
EXEMPLO SIMPLES: RNA RECONHECENDO PORTA AND, PORTA OR



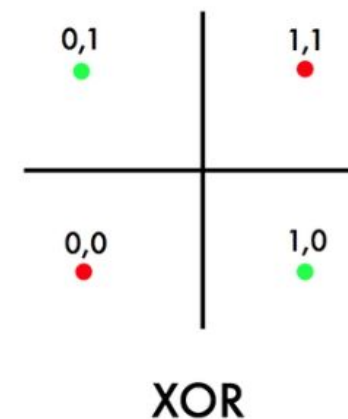
EXPANDINDO COM EMPILHAMENTO DE CAMADAS

As perceptrons não resolvem problemas corriqueiros, como um XOR, mas ao usar duas ou mais camadas resolve. Verique na rede abaixo MLP (MultiLayer Perceptron) se resolve o XOR! Onde não está indicado, considere o peso 1

Uma MLP tem uma camada de entrada, uma ou mais camadas ocultas (hidden) ou intermediárias e uma de saída. Cada camada (excluindo a de saída) tem um neurônio bias e está totalmente conectada à próxima camada. Camadas próximas à entrada (inferiores). Camadas próximas à saída (superiores).



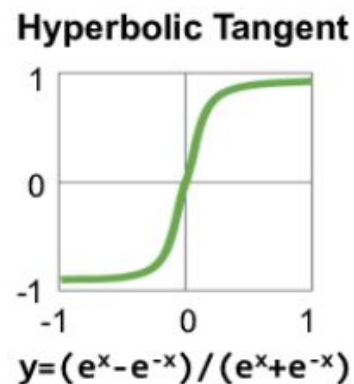
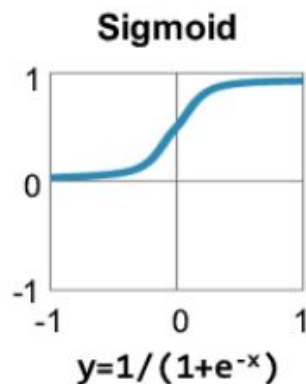
Instâncias não linearmente separáveis



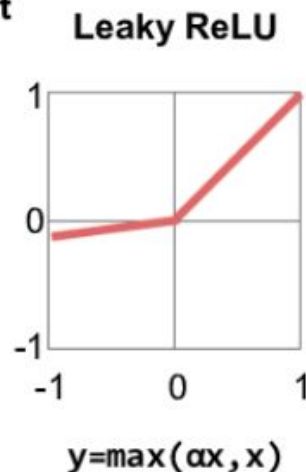
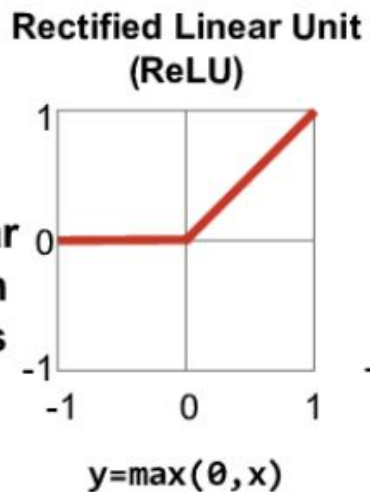
Existem diferentes funções de ativação

FUNÇÕES DE ATIVAÇÃO

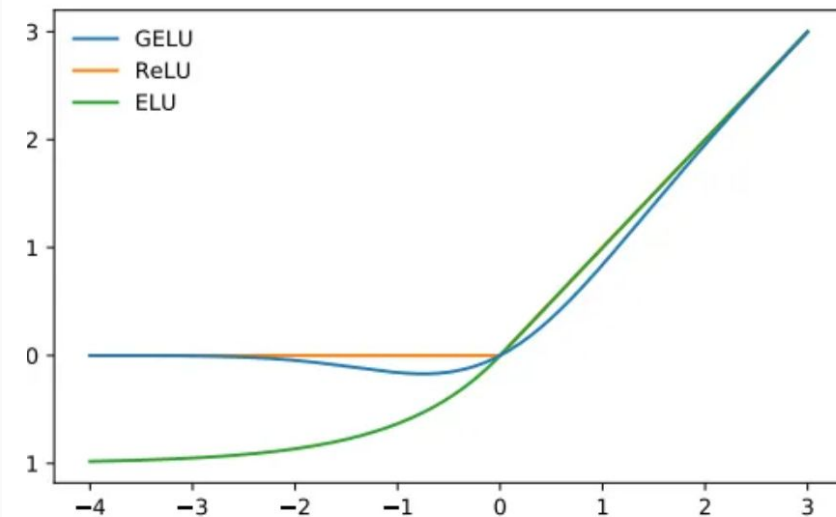
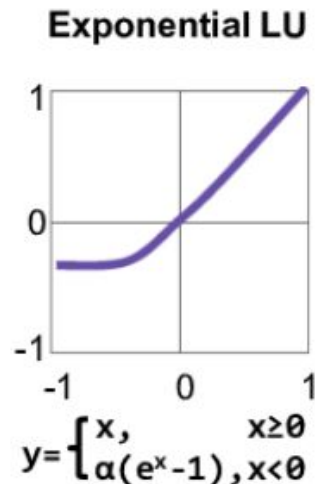
Traditional Non-Linear Activation Functions



Modern Non-Linear Activation Functions



$\alpha = \text{small const. (e.g. 0.1)}$



GELU ($\mu=0, \sigma=1$), ReLU and ELU ($\alpha=1$)

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x)$$
$$\approx 0.5x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715x^3) \right] \right)$$

FUNÇÕES DE ATIVAÇÃO

funções não diferenciáveis em alguns pontos não prejudicam na prática o cálculo do gradiente, pois possuem derivadas laterais definidas

sigmóides e tangente hiperbólica não são boas funções de ativação quando temos muitas camadas na rede, pois como são saturantes podem dificultar o aprendizado via gradiente. Se for necessário usar, a tanh performa melhor.

Em geral, usamos sigmóide ou tangente hiperbólica na última camada (na saída) quando o problema é de classificação

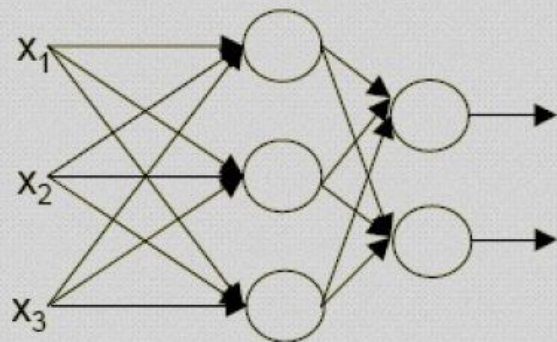
Em outras camadas mais internas (*hidden*) costumamos usar outras funções de ativação como a ReLU (derivada = 1, segunda derivada = 0) ou outras mais especializadas para imagens, que atuam sobre uma função afim (em geral)

PRINCIPAIS ARQUITETURAS

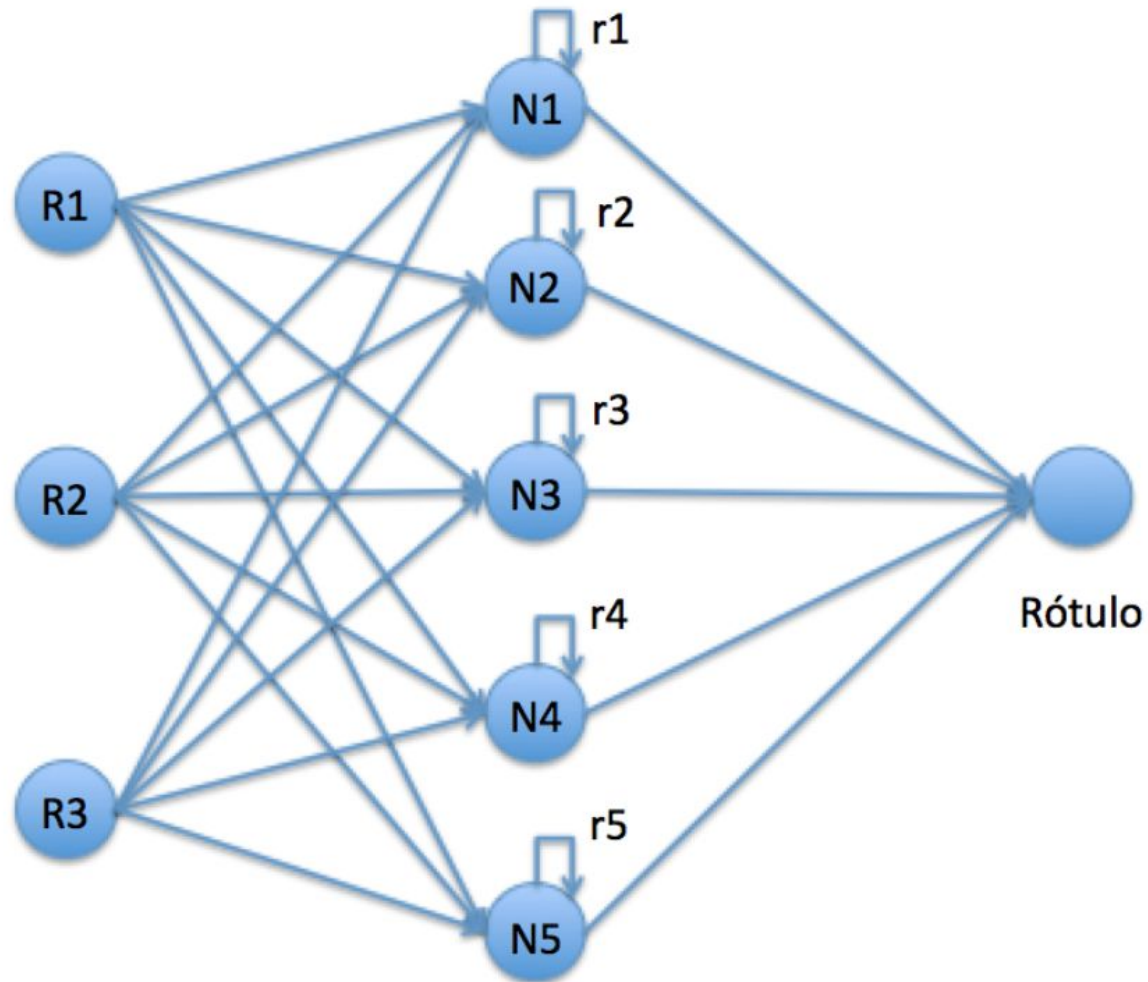
Quanto ao tipo de conexão

Feedforward, ou acíclica

(a saída de um neurônio na i -ésima camada da rede não pode ser usada como entrada de nodos em camadas de índice menor ou igual a i)



PRINCIPAIS ARQUITETURAS



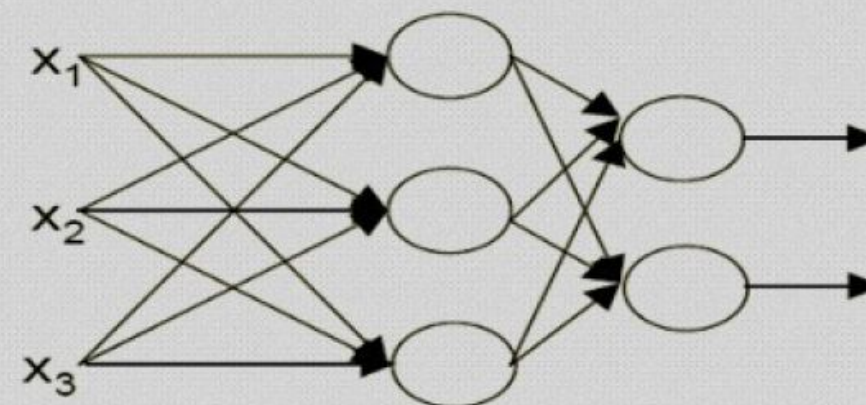
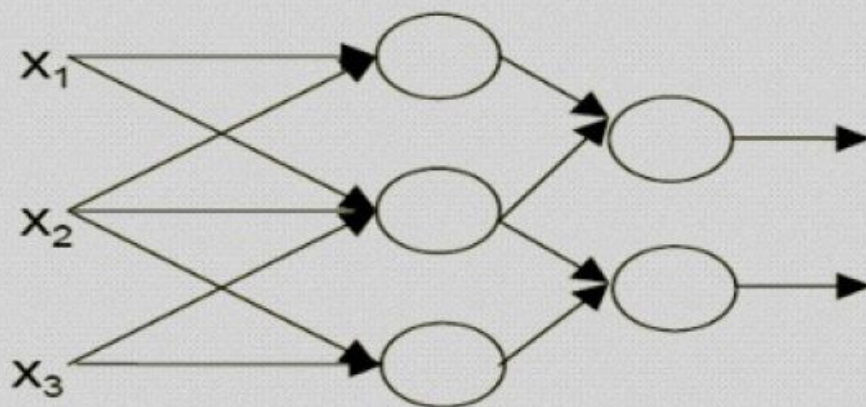
Redes Recorrentes e suas variantes (base PLN, LSTM, GRU): embora superadas pelos Transformers em muitas tarefas PLN, ainda são usadas em dados sequenciais, como séries temporais

PRINCIPAIS ARQUITETURAS

Quanto ao tipo de conectividade

Parcialmente conectada ou Completamente conectada

CNN (sparse networks)



PRINCIPAIS ARQUITETURAS

Quanto à estrutura

Estática

a estrutura **não se altera**, ou seja, o número de neurônios, o número de camadas e o grau de conectividade não se alteram

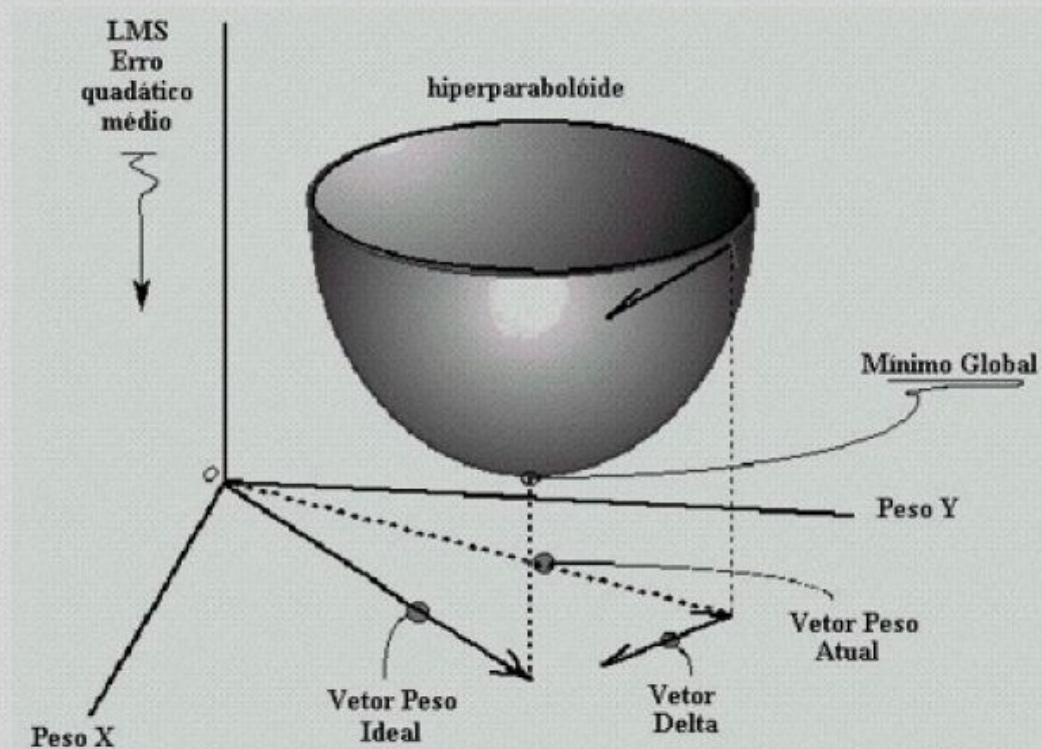
Auto-organizável

são redes em que tanto o número de neurônios como o de camadas **são dinâmicos**

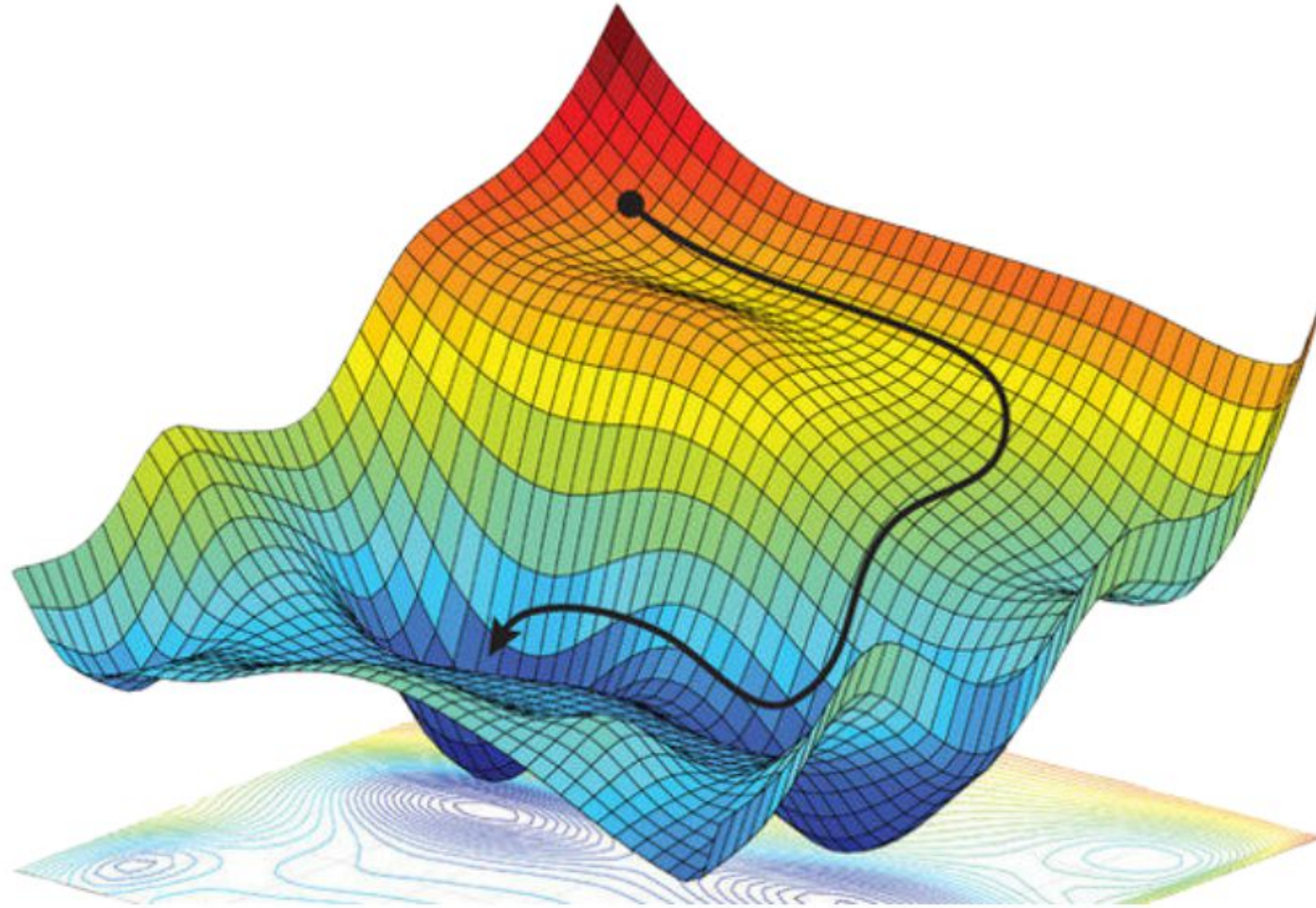
IDEIA GERAL DO TREINAMENTO - BACKPROPAGATION

(depende de derivadas parciais da função de perda em função dos pesos)

Os algoritmos de aprendizagem procuram encontrar o mínimo global da função de erro da rede neural



Solução computacional eficiente para minimização de funções de perda, logo, aprendizado



MODELOS LINEARES DE REGRESSÃO

Machine Learning SUPERVISIONADA: estimar modelos que, embora simplificações da realidade, apresentem a melhor aderência possível entre os valores **reais (observados)** e os valores preditos

Esta é a equação geral da Regressão Linear Múltipla, com n features. Se considerarmos apenas uma, o objetivo é encontrar uma equação que apresente a relação entre uma variável dependente (target) e uma variável explicativa (feature)

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n, \text{ RLM}$$

$$\hat{y} = \theta_0 + \theta_1 x_1, \text{ RLS, onde } \theta_0 : \text{viés, intercepto, coef. linear}$$

x_i : é a i -ésima feature e θ_j é o j -ésimo parâmetro do modelo

Cada parâmetro do modelo (com exceção do intercepto) na verdade reflete o PESO de determinada feature, então vamos adotar uma notação mais comum em aprendizado de máquina, o peso como sendo a letra W

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n = \mathbf{W} \cdot \mathbf{x} = \mathbf{W}^T \mathbf{x}$$

Notação

Medidas de desempenho comum em Regressão

Scikit-Learn.metrics – root_mean_squared_error (scikit-learn >= 1.4)

Raiz do Erro Médio Quadrático:
$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\mathbf{W}^T \mathbf{x}^{(i)} - y^{(i)})^2}$$

Peso maior para grandes erros

Erro Médio Absoluto:
$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\mathbf{W}^T \mathbf{x}^{(i)} - y^{(i)}|$$

Indicado para bases com outliers...

m: número de instâncias no conjunto de dados

$\mathbf{x}^{(i)}$ é um vetor de todos os valores das features (excluindo o rótulo) da **i** – **esima** instância
 $y^{(i)}$ é seu rótulo, valor desejado da saída para aquela instância

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118.20 \\ 33.91 \\ 1416 \\ 38372 \end{pmatrix} \quad y^{(1)} = 156400 \quad \mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^T \\ (\mathbf{x}^{(2)})^T \\ \vdots \\ (\mathbf{x}^{(m)})^T \end{pmatrix} = \begin{pmatrix} -118.29 & 33.91 & 1416 & 38372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

Notação

Medidas de desempenho comum em Regressão

O R-quadrado é uma medida estatística também conhecida como **coeficiente de determinação**, que serve para avaliar a existência de uma relação útil entre a variável dependente (Y) e as variáveis independentes (Xi) em um modelo de regressão.

Ele representa a proporção da variância de Y que foi explicada pelas variáveis independentes no modelo.

Ele fornece uma indicação da qualidade do ajuste e, portanto, uma medida de quão bem as amostras não vistas podem ser previstas pelo modelo, por meio da proporção da variância explicada.

O R-quadrado está sempre entre 0 e 1:

- * 0 indica que o modelo não explica nada da variabilidade dos dados de resposta ao redor de sua média.

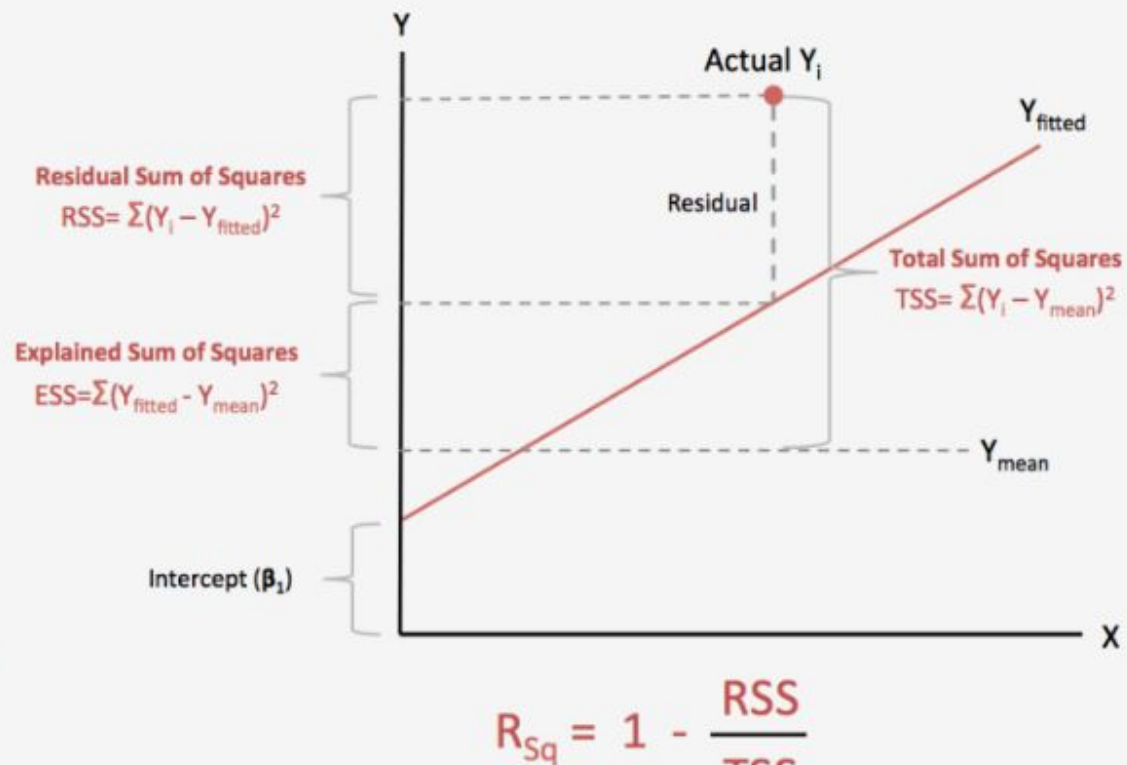
- * 1 indica que o modelo explica toda a variabilidade dos dados de resposta ao redor de sua média.

Em geral, quanto maior o R-quadrado, melhor o modelo se ajusta aos seus dados.

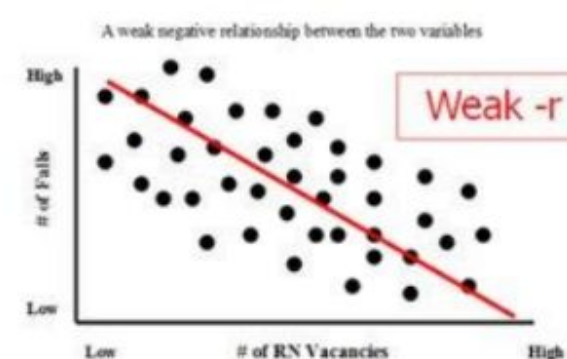
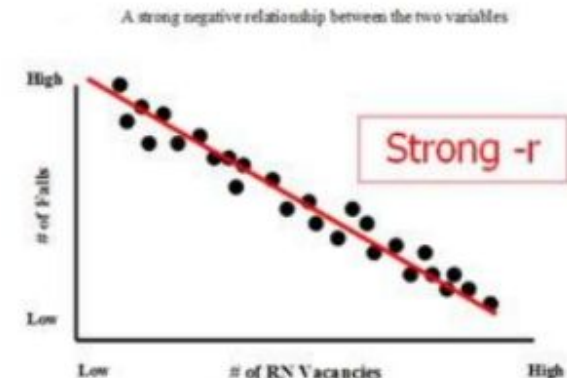
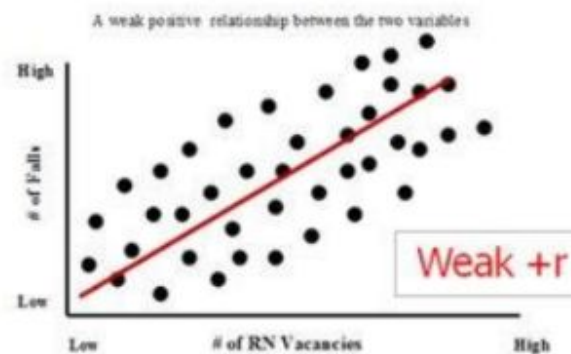
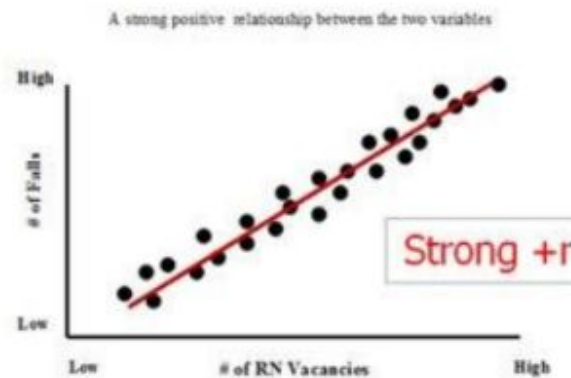
$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Medidas de desempenho comum em Regressão

R-Squared Explanation

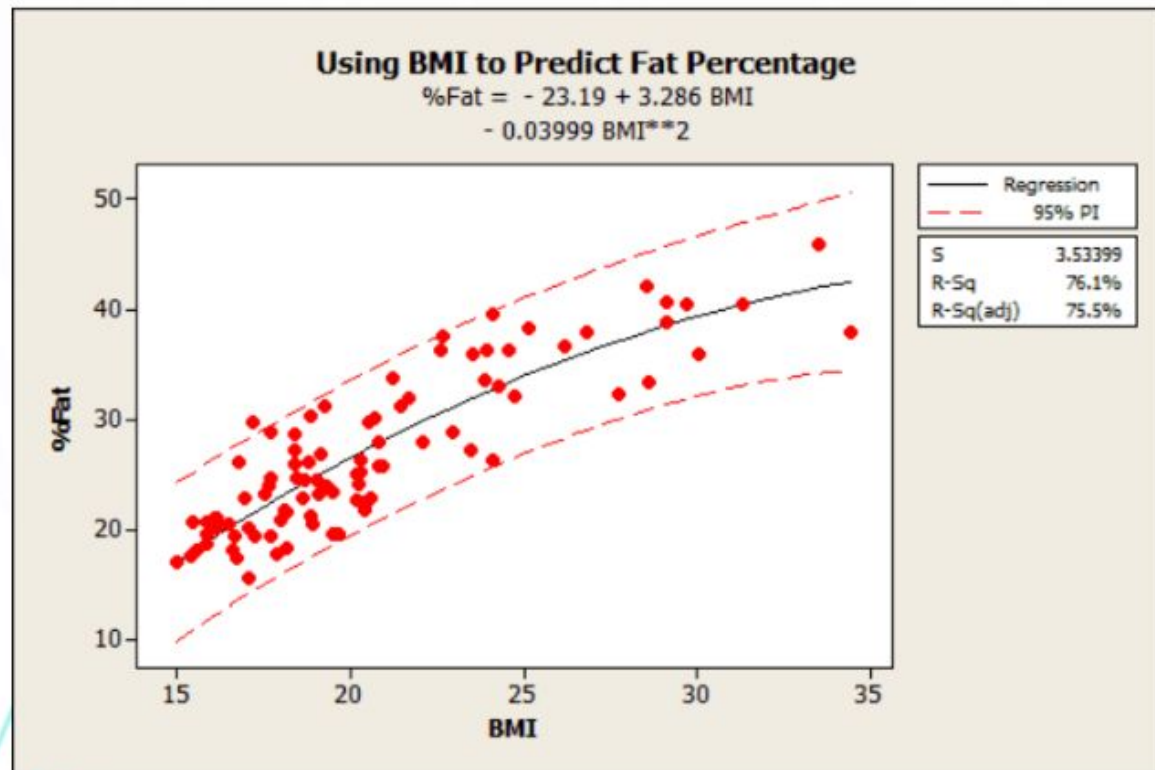


Intervalo de Confiança 95%



Notação

Medidas de desempenho comum em Regressão



Existe uma **probabilidade de 95%** de que, no futuro, o verdadeiro valor do parâmetro da população (por exemplo, média) caia no intervalo **X** (limite inferior) e **Y** (limite superior).

O intervalo de confiança é importante para indicar a margem de incerteza (ou imprecisão) frente a um cálculo efetuado. Esse cálculo usa a amostra do estudo para estimar o tamanho real do resultado na população de origem.

O cálculo de um intervalo de confiança é uma estratégia que considera a amostragem de erro. A dimensão do resultado do seu estudo e seu intervalo de confiança caracterizam os valores presumíveis para a população original.

Quanto mais estreito for o intervalo de confiança, maior é a probabilidade da porcentagem da população de estudo representar o número real da população de origem dando maior certeza quanto ao resultado do objeto de estudo.

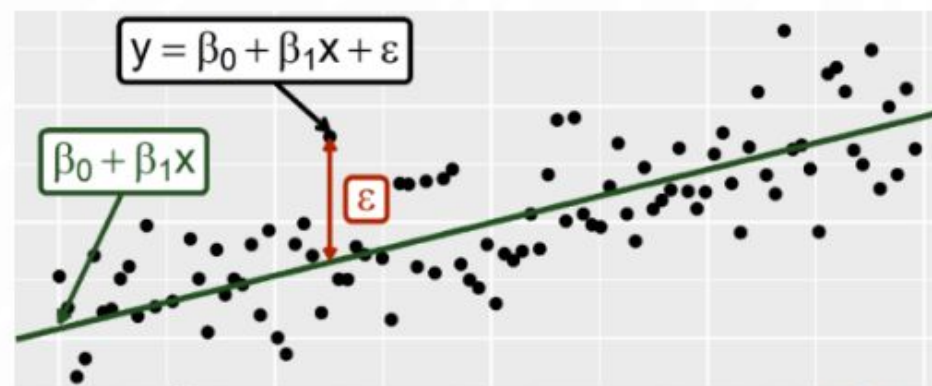
<https://www.significados.com.br/intervalo-de-confianca/>

A obtenção dos pesos do modelo (e do bias) consiste em resolver um problema de otimização de **minimização** da função de perda do erro quadrático médio (mean squared error – MSE) entre os valores observados e os valores preditos com a restrição de que este erro seja nulo

Ou seja, encontrar o vetor de pesos tal que $\min \frac{1}{m} \sum_{i=1}^m \left(\mathbf{W}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$, s.a. $\epsilon = 0$ onde $y = w_0 + w_1 x_1 + \epsilon$

Uma solução é a equação normal $\hat{\mathbf{W}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X} \mathbf{y}$

Mas há uma inversa de produto de matrizes a resolver $(n+1) \times (n+1)$. A classe **LinearRegression** do scikit-learn é baseada na função `scipy.linalg.lstsq()` – mínimos quadrados, que usa uma versão mais efetiva com decomposição de valores singulares (SVD).



Complexidade eq. normal: $O(n^{2.4})$ a $O(n^3)$

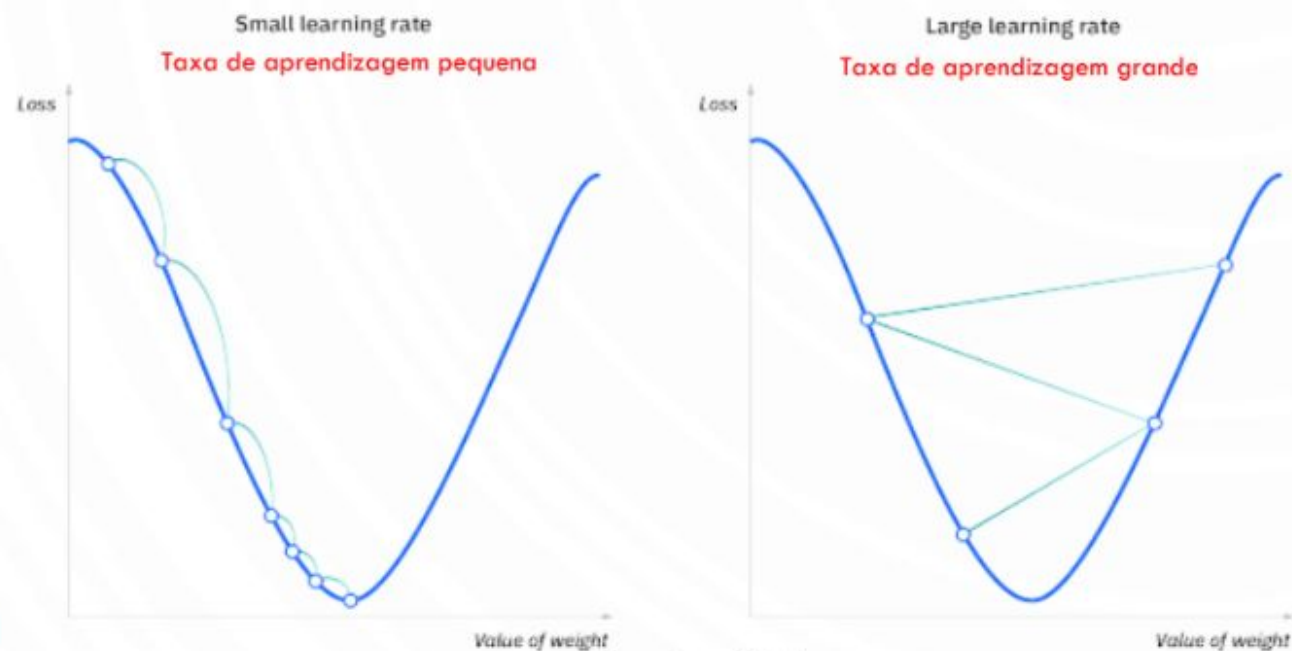
Complexidade SVD scikit-learn: $O(n^2)$

São contudo eficientes para lidar com grandes conjuntos de treinamento: $O(m)$

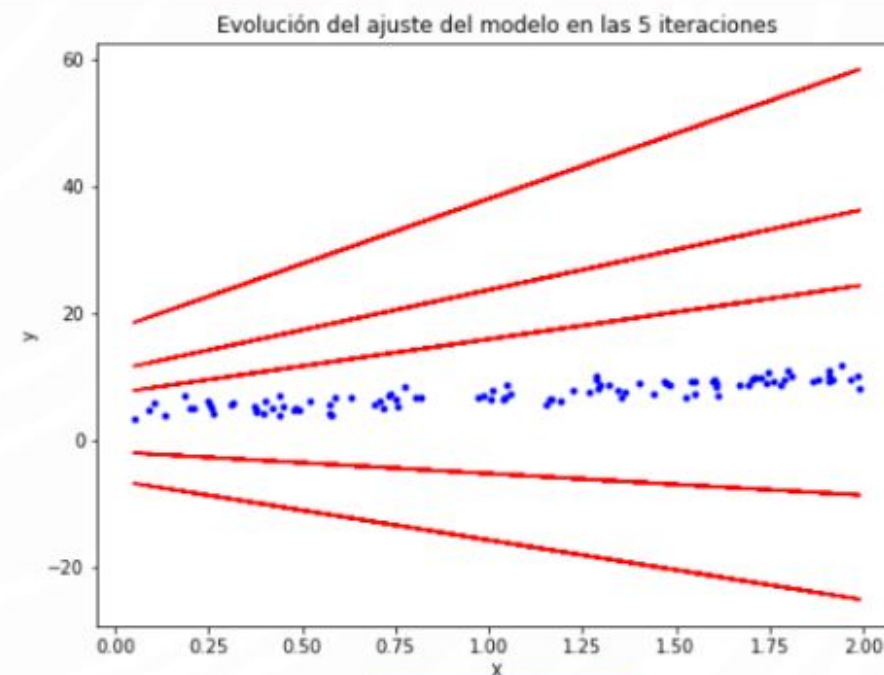
Para predição são lineares para número de instâncias e de características

GRADIENTE DESCENDENTE - GD

O gradiente descendente é um algoritmo de otimização que costuma ser usado para treinar modelos de aprendizado de máquina e redes neurais. Ele treina modelos de aprendizado de máquina minimizando os erros entre os resultados previstos e os reais, ou seja minimizando a função de perda associada.

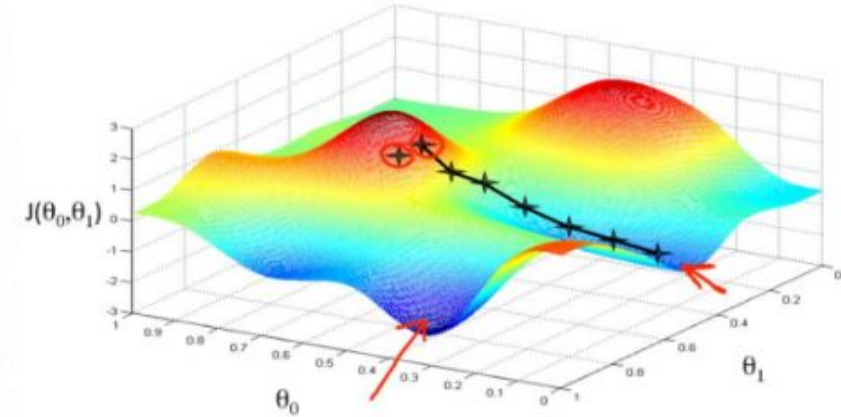
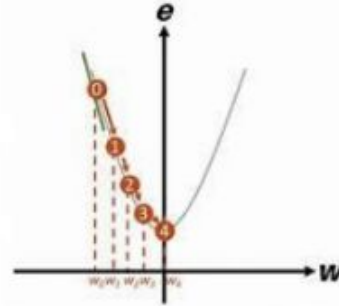
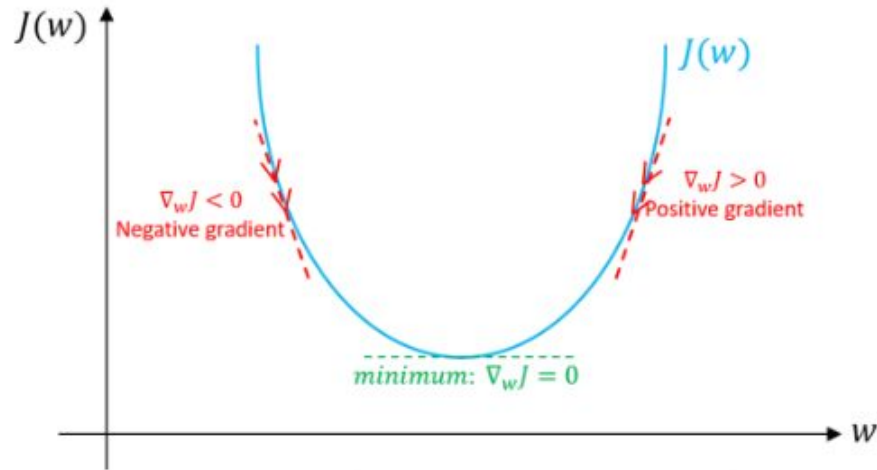


Fonte: IBM



Aprendizado

GRADIENTE DESCENDENTE EM LOTE (BATCH) - GD



É necessário calcular o gradiente da função de perda em relação à cada peso do modelo, ou seja, quanto a função mudará caso se modifique um pouco o peso. A cada etapa TODO o conjunto (lote de dados) é usado, sendo lento para conjunto de treinamento muito grandes. Contudo, escala bem o número de características, sendo muito mais rápido do que a equação normal ou SVD.

$$\frac{\partial}{\partial w_j} MSE(\mathbf{W}) = \frac{2}{m} \sum_{i=1}^m \left(\mathbf{W}^T \mathbf{x}^{(i)} - y^{(i)} \right) \mathbf{x}_j^{(i)}$$

$$\nabla_w MSE(\mathbf{W}) = \begin{pmatrix} \frac{\partial}{\partial w_0} MSE(\mathbf{W}) \\ \frac{\partial}{\partial w_1} MSE(\mathbf{W}) \\ \vdots \\ \frac{\partial}{\partial w_n} MSE(\mathbf{W}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{XW} - \mathbf{y})$$

Expressão de aprendizado – etapa do GD, supondo iterador k :

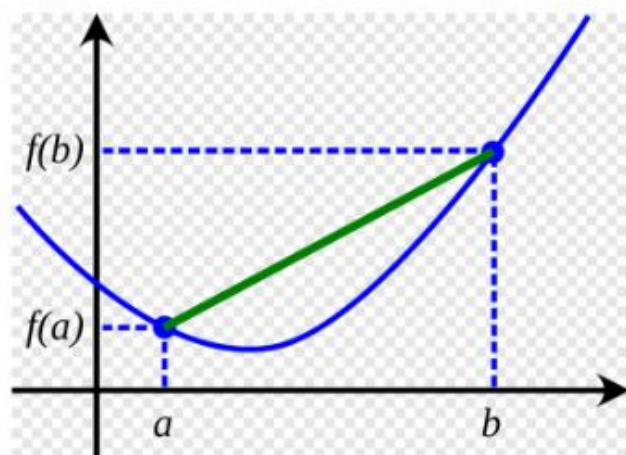
$$\mathbf{W}(k+1) = \mathbf{W}(k) - \eta \nabla_{\mathbf{w}} \text{MSE}(\mathbf{W}), \text{ onde } \eta: \text{ taxa de aprendizagem}$$

Taxa de aprendizagem baixa: muitas iterações para convergir, lento

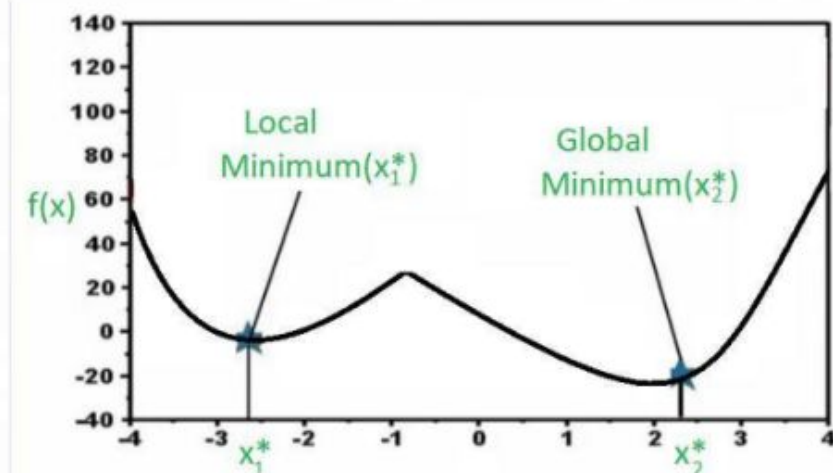
Taxa de aprendizagem alta: pode atravessar o vale e ir para o outro lado em lugar ainda mais alto do que onde estava, divergindo

Com funções convexas (como MSE) se pressupõe que não há mínimos locais, apenas um mínimo global e não muda abruptamente. A BUSCA é feita no espaço de pesos do modelo. **Quanto mais pesos, mas complexa a busca.**

Contudo é muito importante que as features do conjunto de treinamento tenham a mesma escala!



Dados dois pontos, o segmento de reta que os une nunca cruza a curva



Selecione uma instância **aleatória** no conjunto de treinamento a cada etapa e calcula os gradientes com base apenas nesta instância única

Mais rápido, menos dados a cada iteração

Treinamento em grandes conjuntos de dados, pois só uma precisa estar na memória a cada iteração

Contudo, é aleatório, menos regular que o em batch. Não desce suave, mas fica oscilando, pode gerar solução subótima

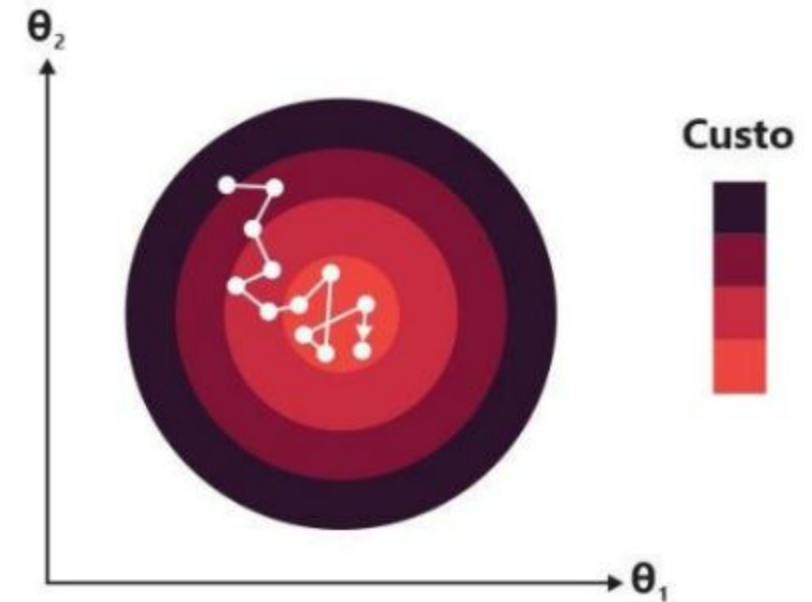
Para funções de perda irregulares (diferente da MSE) pode ajudar a fugir de mínimos locais e chegar ao global

Mas pode nunca se estabilizar no mínimo, e existem soluções de reduzir gradualmente a taxa de aprendizado

Implementar CRONOGRAMA DE APRENDIZADO

No Scikit-Learn, SGDRegressor

Embaralhar o conjunto de treinamento junto com os rótulos, e as instâncias precisam ser independentes



Gradiente Descendente Estocástico

GRADIENTE DESCENDENTE MINI-BATCH

Calcula os gradientes com base em pequenos conjuntos aleatórios de instâncias

Permite obter ganho de desempenho com a otimização de hardware nas operações da matriz, especialmente se usar GPU

O progresso do algoritmo é menos irregular que o GD estocástico, principalmente com grandes mini-batches

Pode ser mais difícil escapar de mínimos locais

- O caminho do batch para no mínimo, SGD e mini-batch prosseguem, perto do mínimo.
- GD batch leva muito tempo para cada época, os outros também alcançariam com um bom cronograma de aprendizado

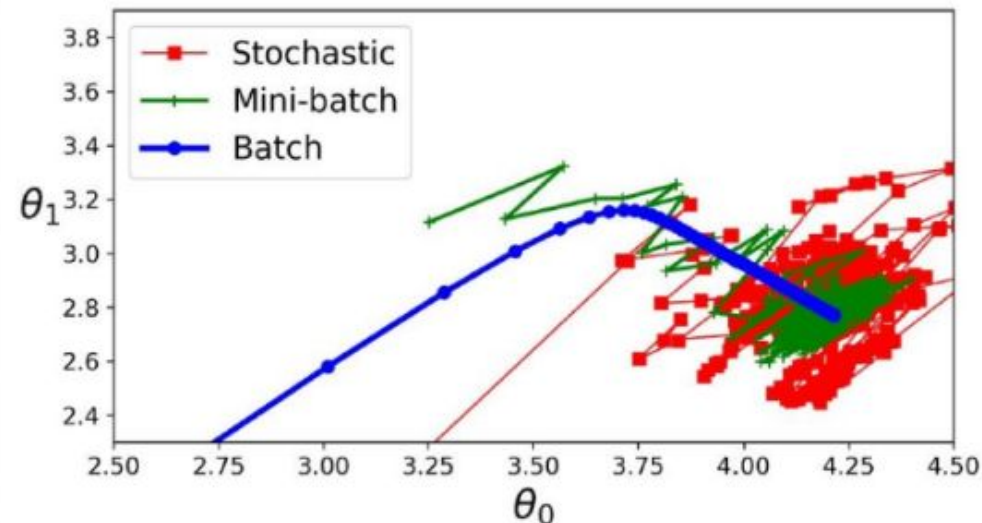


Figure 4-11. Gradient Descent paths in parameter space

(Géron, 2019)

No BACKPROPAGATION este gradiente é calculado reversamente pelo método autodiff reverso (com *chain rule*), gerando um grafo computacional reaproveitável

Revisão rápida de funções compostas

Uma função é *composta* se você puder escrevê-la como $f(g(x))$. Em outras palavras, é uma função dentro de uma função ou uma função de uma função.

Por exemplo, $\cos(x^2)$ é composta, porque se considerarmos $f(x) = \cos(x)$ e $g(x) = x^2$, então $\cos(x^2) = f(g(x))$.

g é a função dentro de f , então chamamos g de função "interna" e f de função "externa".

$$\underbrace{\cos(\overbrace{x^2}^{\text{interna}})}_{\text{externa}}$$

No BACKPROPAGATION este gradiente é calculado reversamente pelo método *autodiff* reverso, gerando um grafo computacional reaproveitável

Chain rule:

Se $y = f(g(x))$, e $u = g(x)$

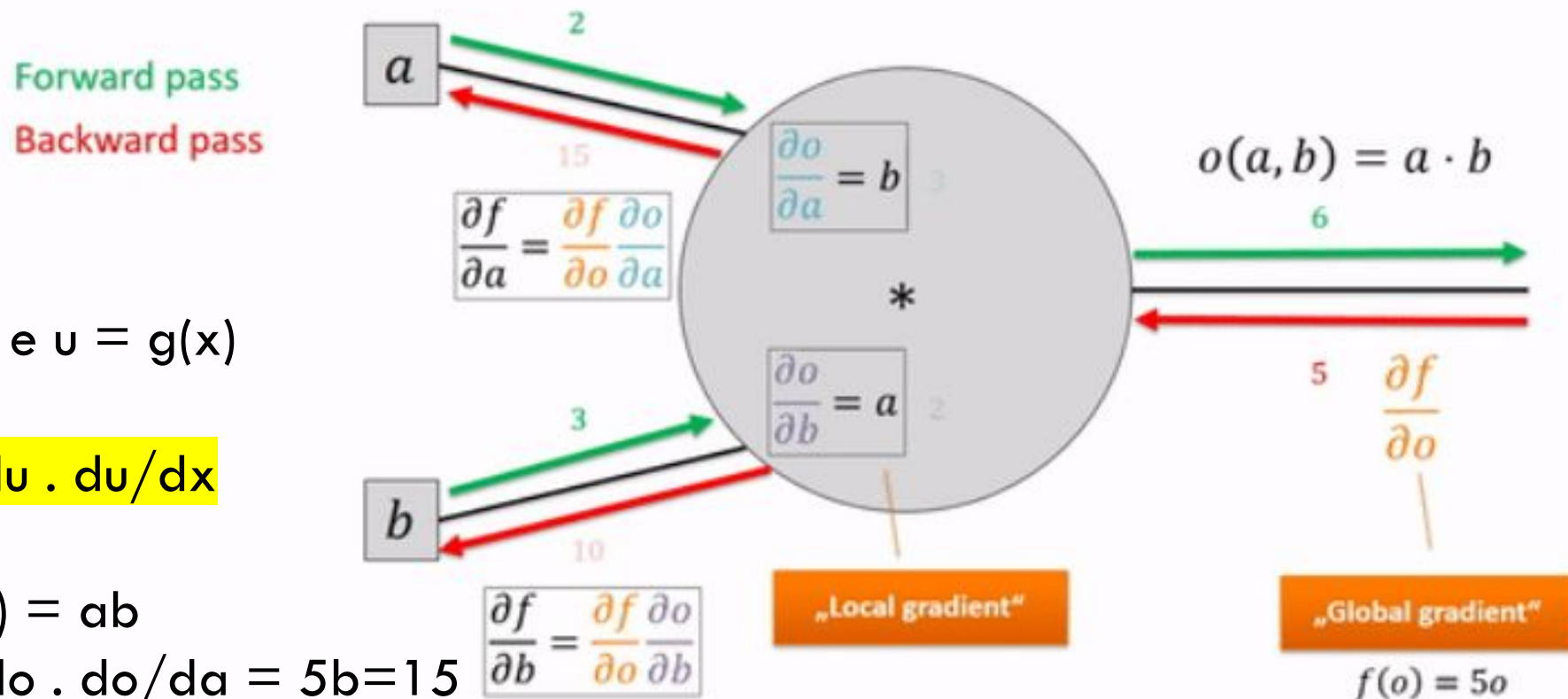
Então $y = f(u)$

$$dy/dx = dy/du \cdot du/dx$$

$y = f(o)$, $o(a,b) = ab$

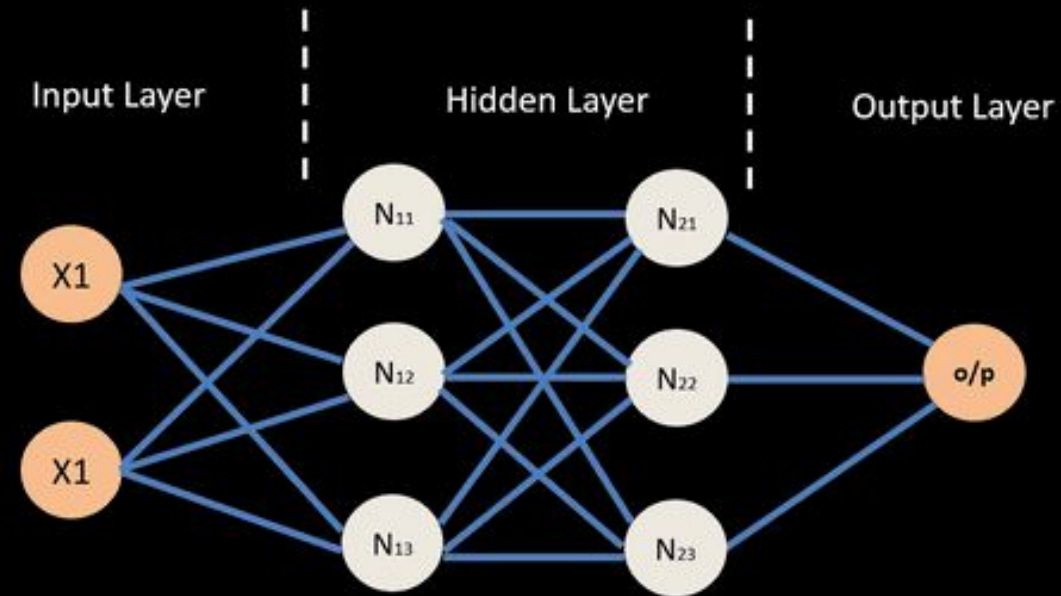
$$dy/da = dy/do \cdot do/da = 5b = 15$$

$$dy/db = dy/do \cdot do/db = 5a = 10$$



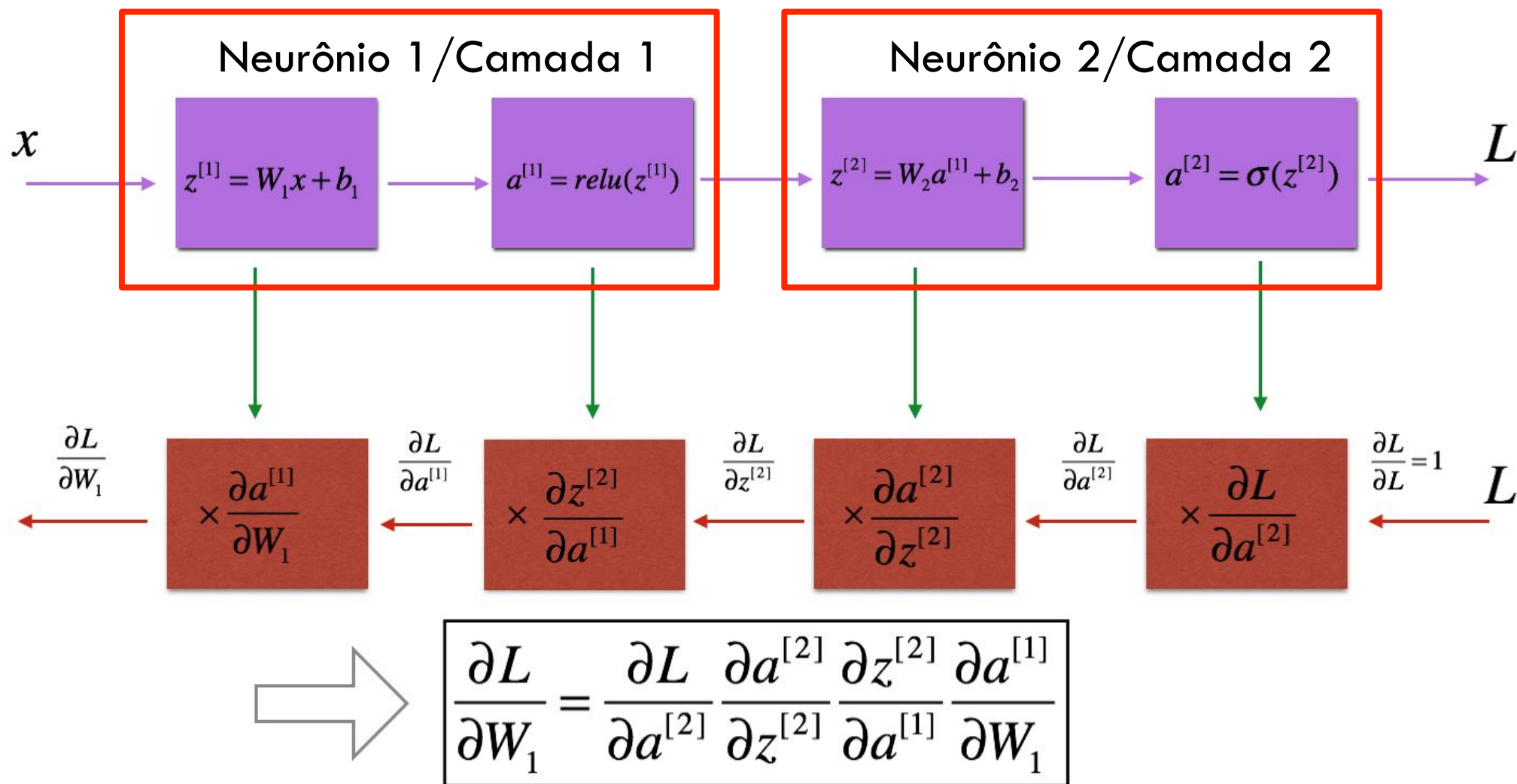
Fonte

Neural Network – Backpropagation



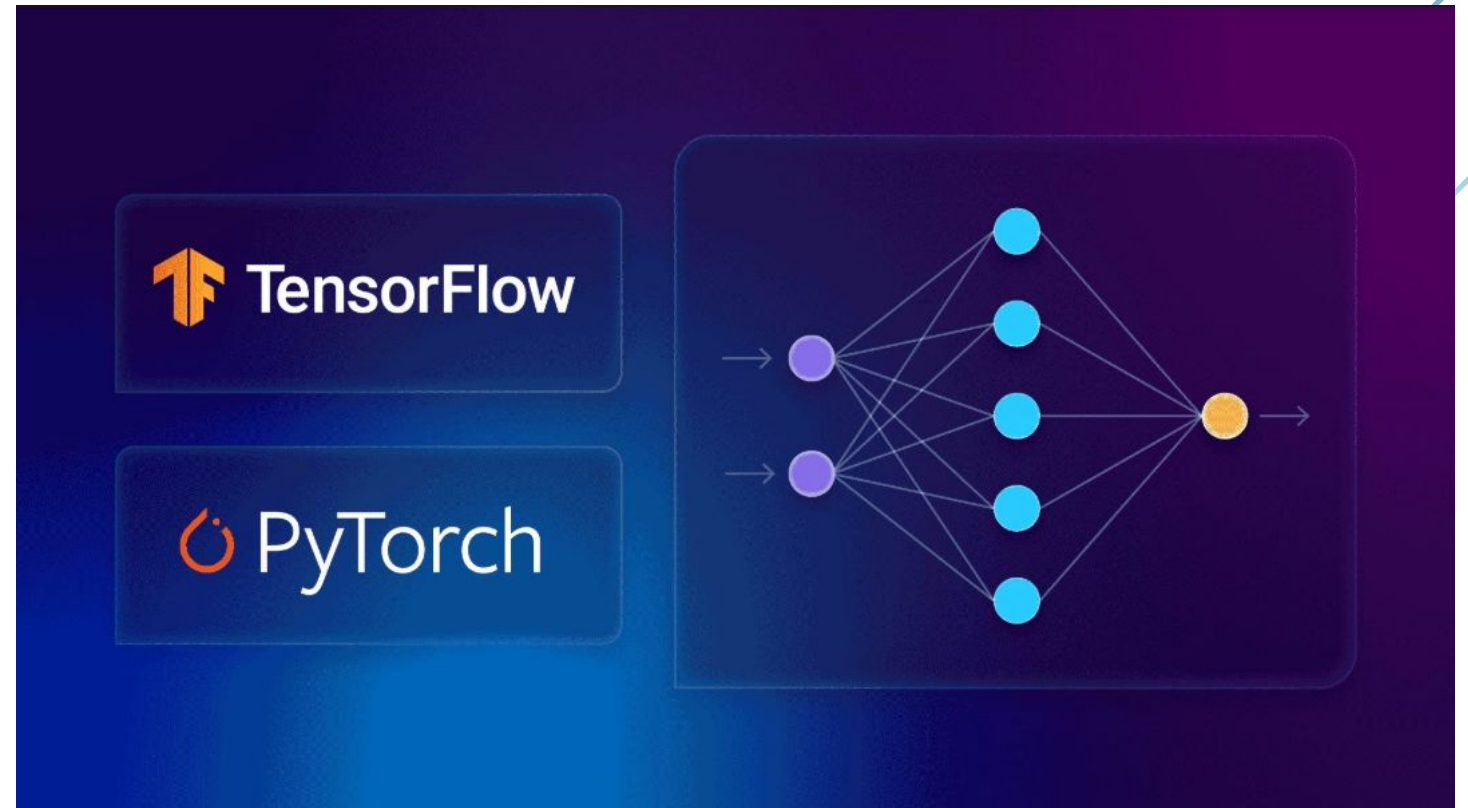
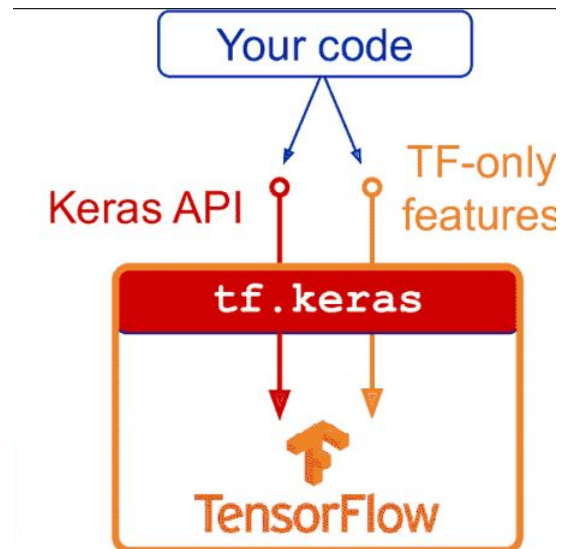
© machinelearningknowledge.ai

Suponha que deseja-se a derivada parcial de L (loss) em função do peso W_1



FRAMEWORKS

<https://keras.io/api/>



<https://docs.pytorch.org/docs/stable/index.html>

Meta AI (2016), atualmente Linux Foundation (torch)

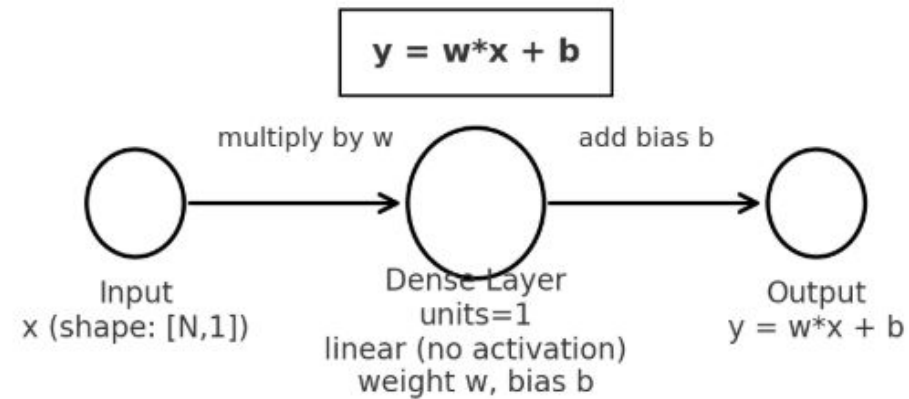
[tensorflow.org](https://www.tensorflow.org/)

Google Brain, 2015 (sucessor do DistBelief de 2011). Google Brain hoje é Google AI

EXEMPLO - TS

```
import numpy as np
from tensorflow import keras
model = keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')
x = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
y = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
model.fit(x, y, epochs=100)
print(model.predict(np.array([10.0])))
```

Arquitetura: Rede Linear (1 input → Dense(1) → 1 output)



EXEMPLO - TORCH

```
import torch https://www.geeksforgeeks.org/deep-learning/pytorch-connection-between-lossbackward-and-optimizerstep/
import torch.nn as nn
import torch.optim as optim

# Dados (tensores do PyTorch)
x = torch.tensor([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0]).unsqueeze(1) #
shape (N, 1)
y = torch.tensor([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0]).unsqueeze(1)

# Modelo equivalente ao Dense(units=1, input_shape=[1])
model = nn.Linear(in_features=1, out_features=1)

# Loss e otimizador
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# Treinamento (100 epochs)
epochs = 100
```

EXEMPLO

```
for epoch in range(epochs):  
    # forward  
    y_pred = model(x)  
    loss = criterion(y_pred, y)  
  
    # backward  
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()  
  
    # opcional: print para acompanhar  
    if (epoch+1) % 20 == 0:  
        print(f"Epoch {epoch+1}: loss = {loss.item():.4f}")  
  
    # Predição para x=10  
    with torch.no_grad():  
        prediction = model(torch.tensor([[10.0]]))  
    print("Predição para x=10:", prediction.item())
```

EXEMPLO 2 - quantos parâmetros tem esta arquitetura?

```
model = keras.Sequential()
model.add(keras.layers.Dense(8))
model.add(keras.layers.Dense(4))
model.build((None, 16)) #entrada vetor size 16
```

```
5 model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 8)	136
dense_5 (Dense)	(None, 4)	36

Total params: 172 (688.00 B)
Trainable params: 172 (688.00 B)
Non-trainable params: 0 (0.00 B)

Quem está certo?

O cálculo para cada camada é o seguinte:

1. Primeira Camada Densa (8 unidades)

- Entrada: 16
- Saída (unidades): 8
- Cálculo: $(16 \times 8) + 8 = 128 + 8 = 136$ parâmetros.

2. Segunda Camada Densa (4 unidades)

- Entrada: A saída da camada anterior é a entrada desta, ou seja, 8.
- Saída (unidades): 4
- Cálculo: $(8 \times 4) + 4 = 32 + 4 = 36$ parâmetros.

Total de Parâmetros

O número total de parâmetros do modelo é a soma dos parâmetros de todas as camadas:

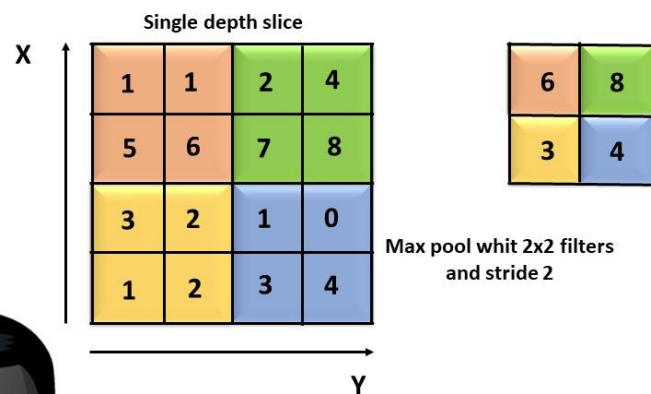
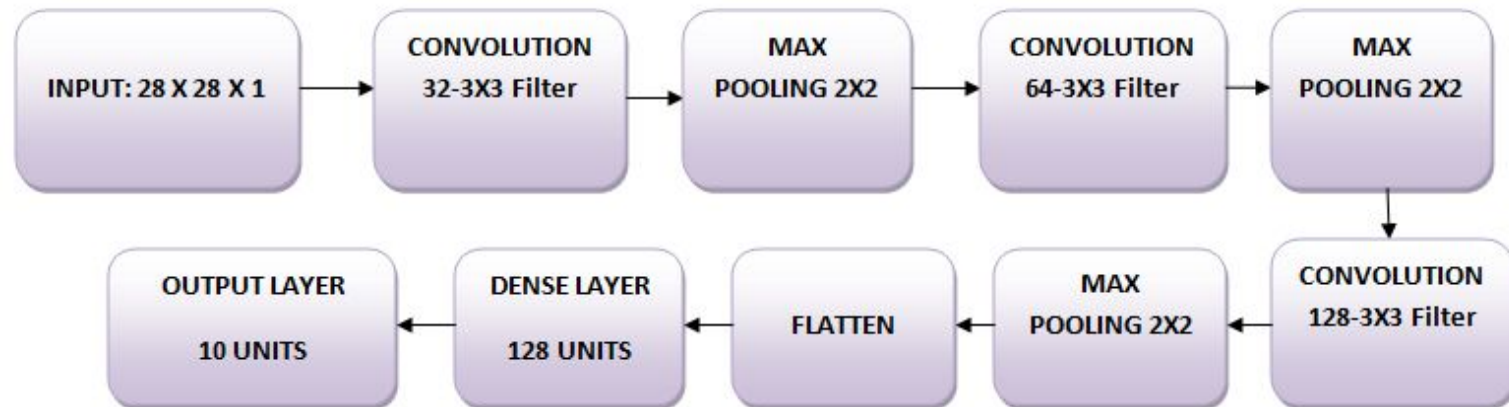
$$\text{Total de Parâmetros} = 136 + 36 = 172$$

Você pode verificar essa contagem usando o método `model.summary()` no Keras após a chamada `model.build()`.

E uma rede convolucional para o MNIST 10 classes?

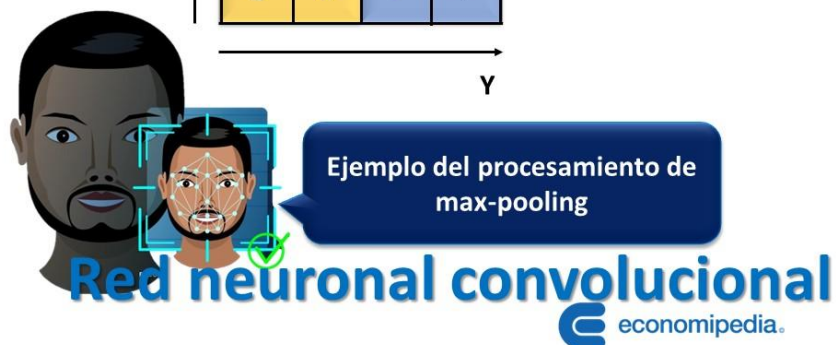
```
model = keras.Sequential()
model.add(keras.layers.Conv2D(32, kernel_size=(3, 3), activation='linear', input_shape=(28, 28, 1), padding='same'))
model.add(keras.layers.LeakyReLU(alpha=0.1))
model.add(keras.layers.MaxPooling2D((2, 2), padding='same'))
model.add(keras.layers.Conv2D(64, (3, 3), activation='linear', padding='same'))
model.add(keras.layers.LeakyReLU(alpha=0.1))
model.add(keras.layers.MaxPooling2D(pool_size=(2, 2), padding='same'))
model.add(keras.layers.Conv2D(128, (3, 3), activation='linear', padding='same'))
model.add(keras.layers.LeakyReLU(alpha=0.1))
model.add(keras.layers.MaxPooling2D(pool_size=(2, 2), padding='same'))
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(128, activation='linear'))
model.add(keras.layers.LeakyReLU(alpha=0.1))
model.add(keras.layers.Dense(10, activation='softmax'))
model.summary()
```

E uma rede convolucional para o MNIST 10 classes?



O kernel é uma janela deslizante de um tamanho específico (comumente 2x2 ou 3x3 pixels) que percorre a imagem de entrada. Em cada posição que o kernel cobre, a operação de *max pooling* seleciona o valor máximo (mais alto) dos pixels dentro dessa janela e o passa para a próxima camada da rede neural.

O stride é um parâmetro que define a distância, em pixels, que o kernel se move horizontalmente e verticalmente após cada operação de pooling.



- 1) Playground Tensorflow - obter feeling...(estudo dirigido)
- 2) Otimizar MNIST no conjunto de teste
- 3) Construir a melhor arquitetura
 - a) para os dígitos 0..9
 - b) para o V e F
 - c) para as letras do artigo **silvafilho2022** sobre o multiprova corretor (elaborar prompt para resumir artigo) - EMNIST
 - d) exportar os modelos (.keras) e analisar o número de parâmetros treináveis e o tamanho (MB) do modelo
 - e) Como reduzir para mobile?